

Applicable Release:
SAP Fiori for Request Approvals 1.0 SP 04

Document Version: 1.1 – 2020-07-21

Extending SAP Fiori My Inbox



Typographic Conventions

Type Style	Description
Example	Words or characters quoted from the screen. These include field names, screen titles, pushbuttons labels, menu names, menu paths, and menu options. Textual cross-references to other documents.
Example	Emphasized words or expressions.
EXAMPLE	Technical names of system objects. These include report names, program names, transaction codes, table names, and key concepts of a programming language when they are surrounded by body text, for example, SELECT and INCLUDE.
Example	Output on the screen. This includes file and directory names and their paths, messages, names of variables and parameters, source text, and names of installation, upgrade and database tools.
Example	Exact user entry. These are words or characters that you enter in the system exactly as they appear in the documentation.
<Example>	Variable user entry. Angle brackets indicate that you replace these words and characters with appropriate entries to make entries in the system.
EXAMPLE	Keys on the keyboard, for example, F2 or ENTER .

Document History

Version	Date	Change
1.0	2015-01-30	Initial Version
1.1	2020-07-21	Minor Update

Table of Contents

1	Business Scenario	6
2	Background Information	10
3	Prerequisites	11
3.1	Technical Requirements	11
3.2	RFC Function Modules.....	11
3.3	DDIC Structures	12
3.3.1	DDIC Structures Required for the RFC Function Modules.....	12
3.3.2	DDIC Structures Required for the Service Queries.....	14
4	Extending in the Gateway	16
4.1	Extending the Task Gateway Service	16
4.1.1	Create a New Service	16
4.1.2	Add Custom Properties to the Service	18
4.1.3	Add the Chain of Approvers to Service	19
4.1.4	Add the Shopping Cart Line Item List to the Service	20
4.1.5	Enable Item-Level Approval	22
4.2	Extending the Service Implementation	24
4.2.1	Retrieve Business Data for the Task List (S2 Screen).....	25
4.2.2	Retrieve Business Data for the Task Details, Task Items, and Approver Chain (S3 Screen)	26
4.2.3	Retrieve Business Data for Item Details (S4 Screen)	28
4.2.4	Enable Item-Level Approval	30
4.3	Configuring the Service in the Gateway	31
4.3.1	Add and Configure the Customer Service	31
4.3.2	Assign Provider to Data Model	31
4.4	Troubleshooting in the Gateway System.....	32
5	Extending the Front End (UI5 Code Extension)	33
5.1	Create an Extended App Skeleton	33
5.1.1	Create a New Project	33
5.1.2	Upload the New Project.....	35
5.1.3	Running in Standalone Mode.....	35
5.2	Add New Fields to the List (S2) and Detail (S3) Screens	37
5.2.1	Define the View Extensions in the Component.js	37
5.2.2	Implement the View Extensions	38
5.3	Create Custom Sorting, Grouping, and Filtering.....	41
5.3.1	Create Custom Grouping	41
5.3.2	Create Custom Filtering	42
5.3.3	Create Custom Sorting	44
5.4	Add a List of Line Items to the Detail Screen (S3).....	44
5.4.1	Define the S3 View Extensions in Component.js.....	45
5.4.2	Extend the S3 View with the List of Line Items	45
5.4.3	Extend the S3 Controller to Enable Item-Level Selection and Approval	46
5.5	Extend the Application with S4 Screen.....	49

5.5.1	Define the Navigation Route to the S4 Screen	49
5.5.2	Create the S4 View	50
5.5.3	Create the S4 Controller	52
5.5.4	Create Formatters	55
5.6	Add a New Tab to the Detail Screen	56
5.6.1	Extend the S3 View with a New Tab	56
5.6.2	Declare the New Tab	57
5.7	Enable Item-Level Approval	58
5.7.1	Change the Action Buttons	58
5.7.2	Implement Event Handler for the Send Button	59
5.7.3	Implement the performCreate Function	61
5.8	Limit the Forward Agent List Size	62
6	Provide Extensions for a Specific Task Type in Detail View	64
6.1	Create Mapping of Task Types with the New View	64
6.2	Extend the S2 Controller	65
6.2.1	Create the S2 Controller	65
6.2.2	Provide Controller Extension for S2 in the Component.js	66
6.3	Add a New View for a Specific Task Type	67
6.3.1	Create a New View for the Task Type	67
6.3.2	Create a New Controller for the Task Type	67
6.3.3	Map the New View in Component.js	69
6.4	Add Extension for the Specific Task Type	70
6.4.1	Write the New Extension Fragment	70
6.4.2	Provide Extensions for the New Fragment in the Component.js	71
6.5	Extend the S3_additionDetails Controller	71
6.5.1	Write the S3_additionDetails Controller Extension	71
6.5.2	Define S3_additionalDetails Controller Extension in the Component.js	72
7	Configuring the Launchpad	75
7.1	Create Target Mapping	75
7.2	Create an App Launcher Tile	76
Appendix		77

1 Business Scenario

With the My Inbox application, you can make important decisions via mobile or desktop devices anywhere and anytime. Using this app, you can process your standard and custom workflow tasks based on the decision options defined in the back-end system.

All Items Inbox

With the My Inbox app, we deliver a pre-configured All Items tile in the SAP Fiori Launchpad, which enables you to easily process all your tasks.

Scenario-Specific Inbox

The My Inbox application also offers you the possibility to define your own workflow scenarios and create scenario-specific tiles in the SAP Fiori Launchpad.

Key Features

- You can process tasks from your SAP Business Workflow, SAP Business Process Management (BPM), and 3rd party providers.
- You can define the decision options you want to display (for example, approve or reject).
- You can perform the standard actions (for example, claim, release, or forward a request).

These actions are available by default; no customizing is needed in the back-end system.

- You can perform mass actions, that is, process several tasks of the same type at the same time.
- You can view and add comments.
- You can view custom attributes that provide additional information about the task instance.
- You can view, upload, and delete attachments.
- You can create and manage substitution rules to manage the tasks in your absence.
- You can send emails or post tasks on a pre-configured JAM site.
- You can view the business cards of the application users.
- You can browse, sort, filter, and group tasks requiring action.
- You can extend the standard application to cover additional UI or functionality needs.
- You can easily process all your tasks with the pre-configured All Items tile.

The goal of this cookbook is to help developers to extend the standard My Inbox application with a Shopping Cart Approval scenario. Through this example scenario, you can learn about the My Inbox-specific extensibility capabilities.

This document is aimed at developers working at the customer's site.

Info

Please keep in mind that this document is not a standard Extensibility Guide, covering all the possible extensibility options. It is a how-to-guide that aims to help developers to extend the standard My Inbox application with a custom approval scenario. It guides you through an example implementation of extending the My Inbox application for the Shopping Cart Approval scenario.

Note

The annotation-based task UI in My Inbox does not support the extensions described in this document.

For example, they are not supported for the following task types UIs with annotation-based task UI:

- Release Purchase Requisition
- Release Purchase Requisition Item
- Release Purchase Contract
- Release Purchase Order
- Review Requirement Document
- Approve Earmarked Funds
- Approve Risk Assessment
- Approve Travel Request
- Approve Trip

You can continue using the annotation-based task UI without the respective extensions or create a custom task UI.

For more information about the general extensibility capabilities, visit the standard product documentation on the Help Portal, at <http://help.sap.com/fiori>.

This document consists of the following parts, describing the main tasks of extending the standard application:

- Extending the Task Gateway Service
- Extending the Front End
- Configuring the Launchpad

Additionally, you can find the complete source code of the extended application.

By following the steps in the upcoming chapters you will extend your standard application in the following way:

Add the following custom fields on the list screen (S2):

- Value
- Currency
- Approval on Item Level
- Shopping Cart (SC) Number

-
- Delivery Date

Add the following information on the detail screen (S3):

- Chain of approvers
- List of line items

Enable the following create operation:

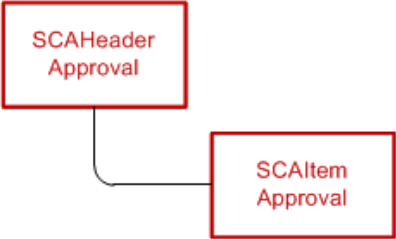
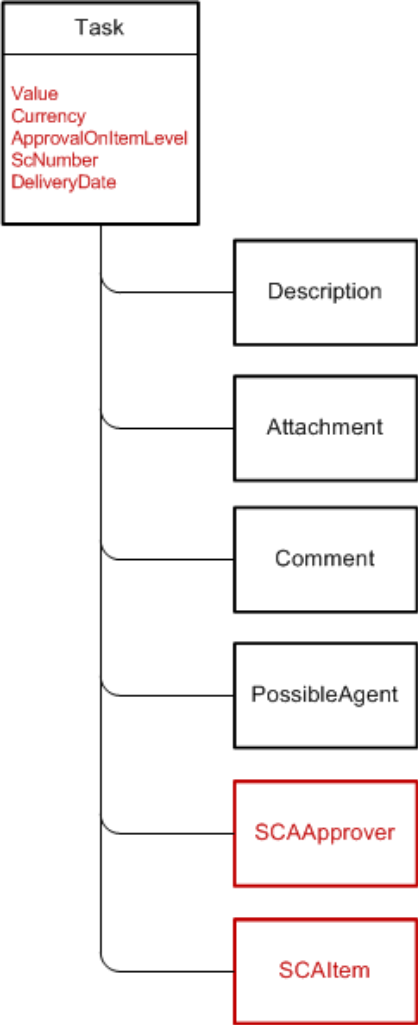
- Item-level approval

The standard Task Gateway Service is only capable of approving work items on header level. In this example, each shopping cart item requires an individual decision, and this cannot be done using the standard `Decision` function import.

In this example (shopping cart) scenario, we extend the standard app, so that it is capable of both header-level and item-level approval.

In case of header-level approval, the [Approve](#) and [Reject](#) buttons are displayed (based on your customizing). However, in case of item-level approval, instead of the Approve and the Reject buttons a [Send](#) button is displayed in the footer bar.

The following diagram shows the standard entities and the relation between them in black. The new, custom entities and properties are shown in red.



2 Background Information

In this example, the following systems are involved:

- Back-end system(for example, SRM)

The back-end system provides the required workflows and business data. Data is transferred from the back end via remote function calls (RFCs).

In the current example, only one back end is involved. This is the recommended scenario for My Inbox back-end extensibility.

- Gateway

The Gateway system provides the Task Gateway Service, which retrieves the business data from the back-end system and feeds it to the front-end application.

- Front-end application (SAPUI5)

The front end is a client-side web application that is built on the JavaScript-based SAPUI5 framework. This cross-device and cross-browser framework supports data binding to consume the data received from the back-end system through the Gateway.

3 Prerequisites

Before you start to extend your My Inbox app, make sure that the prerequisites described in this section are fulfilled:

3.1 Technical Requirements

The following software products must be available in your system landscape:

- Back-end system that contains the business logic
In this example, SRM Shopping Cart Approval 1.0 (GBSRM001)
- UI add-on: FIORI APPROVE REQUESTS X1 1.0 (UIX01CA1 SP04)

For more information about the required back-end and front-end components, see the product documentation at the Help Portal at http://help.sap.com/fiori_bs2013/helpdata/en/69/d21a54c8fd1c6ee10000000a441470/frameset.htm.

You must be familiar with the OData concept.

3.2 RFC Function Modules

To be able to exchange data between your back-end system and Gateway, you need to create a connection between the two systems. You can establish this remote access via web service calls or remote function calls. We recommend using remote function calls (RFCs).

Make sure that that you have RFC function modules in the back end that handles data required for your extensibility scenario.

In this example, we suggest that you create the following RFC function modules in a new function group in your back end:

Technical Name	Name	Description
ZCARTAPP_GET_SC_DETAILS	Get Shopping Cart Details	Retrieves data from the back-end system
ZCARTAPP_GET_WFTASKLIST	Get Shopping Cart Workflow Task List	Retrieves data from the back-end system
ZCARTAPP_ITEMLEVEL_APPROVAL	Shopping Cart Item Level Approval	Performs item-level approval

For more information, about the RFC function modules, see [Appendix A - Source Code of RFC Function Modules for Cart Approval Scenario](#).

3.3 DDIC Structures

3.3.1 DDIC Structures Required for the RFC Function Modules

To enable data exchange, create DDIC structures required for the RFC function modules in the Gateway.

In this example, we suggest that you create the following DDIC structures in your Gateway system:

DDIC Structures for the ZCARTAPP_GET_WFTASKLIST RFC Function Module

- ZCARTAPP_S_APV_SC_WF_LIST

Component	Typing	Component Type	Data Type	Length	Decimal
WORKITEM_ID	1 Types	SWW_WIID	NUMC	12	0
TASK_TYPE	1 Types	HROBJEC_14	CHAR	14	0
SC_GUID	1 Types		RAW	16	0
SC_NUMBER	1 Types	CRMT_OBJECT_ID_DB	CHAR	10	0
SC_NUMBER_FORMATTED	1 Types	CRMT_OBJECT_ID_DB	CHAR	10	0
SC_TYPE	1 Types	CHAR3	CHAR	3	0
APPROVAL_ON_ITEMLEVEL	1 Types		CHAR	1	0
NO_MOB_APPLICATION_PROCEEDING	1 Types		CHAR	1	0
WI_CREATED_AT	1 Types	TZNTSTMP5	DEC	15	0
CREATED_BY_ID	1 Types	SYUNAME	CHAR	12	0
CREATED_BY_NAME	1 Types	CHAR40	CHAR	40	0
ON_BEHALF_OF_ID	1 Types	SYUNAME	CHAR	12	0
ON_BEHALF_OF_NAME	1 Types	CHAR40	CHAR	40	0
REASSIGN_BY_ID	1 Types	SYUNAME	CHAR	12	0
REASSIGN_BY_NAME	1 Types	CHAR40	CHAR	40	0
SUBSTITUTING_FOR_ID	1 Types	SYUNAME	CHAR	12	0
SUBSTITUTING_FOR_NAME	1 Types	CHAR40	CHAR	40	0
SC_DESCRIPTION	1 Types	CRMT_PROCESS_DESCR PTION	CHAR	40	0
VALUE	1 Types		CURR	15	2
CURRENCY	1 Types	WAERS	CUKY	5	0
CURRENCY_APPROVER	1 Types	WAERS	CUKY	5	0
AMOUNT_CONVERSION_TYPE_CODE	1 Types	CHAR1	CHAR	1	0
NR_OF_ITEMS	1 Types		INT4	10	0
STICKED_ITEM_DESCR	1 Types		STRING	0	0
STICKED_ITEM_DESCR_SHORT	1 Types		STRING	0	0
ITEM_DESCRIPTIONS	1 Types		STRING	0	0
SEARCH_FOR_TEXT	1 Types		STRING	0	0
NOTE_COUNT	1 Types	CHAR2	CHAR	2	0
HAS_COMMENTS	1 Types	BOOLE_D	CHAR	1	0

Note

On the [Currency/quantity field](#) tab, enter the following data for the [VALUE](#) field:

Reference table: [ZCARTAPP_S_APV_SC_WF_LIST](#)

Reference field: [CURRENCY](#)

- ZCARTAPP_T_APV_SC_WF_LIST
Line Type: ZCARTAPP_S_APV_SC_WF_LIST

DDIC Structures for the ZCARTAPP_GET_SC_DETAILS RFC Function Module

- ZCARTAPP_S_SCAITEM

Component	Typing	Component Type	Data Type	Length	Decimal
INSTANCE_ID	1 Types		STRING	0	0
ITEM_NUMBER	1 Types		CHAR	10	0
DESCRIPTION	1 Types		CHAR	40	0
VENDOR_ID	1 Types		CHAR	10	0
VENDOR_NAME	1 Types		CHAR	40	0
UNIT	1 Types		CHAR	3	0
UNIT_TEXT	1 Types		CHAR	10	0
QUANTITY	1 Types		STRING	0	0
VALUE	1 Types		DEC	8	2
CURRENCY	1 Types		CHAR	5	0
PRICE	1 Types		DEC	7	2
PRICE_UNIT	1 Types		DEC	16	0
DELIVERY_DATE	1 Types	TIMESTAMP	DEC	15	0
CATEGORY_DESCRIPTION	1 Types		CHAR	40	0

- ZCARTAPP_S_SCAAPPROVER

Component	Typing	Component Type	Data Type	Length	Decimal
INSTANCE_ID	1 Types		STRING	0	0
APPROVER_ID	1 Types		CHAR	12	0
APPROVER_SEQUENCE	1 Types	/IWBEP/SB_ODATA_TY _INT2	INT2	5	0
APPROVER_FIRSTNAME	1 Types		CHAR	40	0
APPROVER_LASTNAME	1 Types		CHAR	40	0
APPROVER_FULLNAME	1 Types		CHAR	80	0
APPROVER_ROLE	1 Types		CHAR	40	0
APPROVAL_STATUS	1 Types		CHAR	60	0
APPROVAL_STATUSCODE	1 Types		CHAR	8	0

- ZCARTAPP_T_SCAITEM
Line Type: ZCARTAPP_S_SCAITEM
- ZCARTAPP_T_SCAAPPROVER
Line Type: ZCARTAPP_S_SCAAPPROVER

3.3.2 DDIC Structures Required for the Service Queries

The client application retrieves task details from the Gateway using the \$expand feature of the OData protocol to read details, attachments, and comments using one query. The following DDIC structures are required by the \$expand logic in the Gateway.

- ZCARTAPP_S_TGW_TASK_EXPAND

Component	Typing	Component Type	Data Type	Length	Decimal
.INCLUDE	1 Types	/IWWRK/S_TGW_TASK_EXPAND		0	0
SC_DESCRIPTION	1 Types		CHAR	40	0
SC_NUMBER	1 Types		CHAR	10	0
APPROVAL_ON_ITEMLEVEL	1 Types		CHAR	1	0
CURRENCY	1 Types		CHAR	5	0
VALUE	1 Types		DEC	8	2
DELIVERYDATE	1 Types	TIMESTAMP	DEC	15	0

- ZCARTAPP_S_TGW_EXPAND_DEEP

Component	Typing	Component Type	Data Type	Length	Decimal
.INCLUDE	1 Types	ZCARTAPP_S_TGW_TASK_EXPAND		0	0
CREATEDBYDETAILS	1 Types	/IWWRK/S_TGW_USER_DET_EXPAND		0	0
PROCESSORDETAILS	1 Types	/IWWRK/S_TGW_USER_DET_EXPAND		0	0
DESCRIPTION	1 Types	/IWWRK/S_TGW_DESC_EXPAND		0	0
UIEXECUTIONLINK	1 Types	/IWWRK/S_TGW_UI_EXEC_EXPAND		0	0
POSSIBLEAGENTS	1 Types	/IWWRK/T_TGW_POS_AGENT_EXPAND		0	0
PROCESSINGLOGS	1 Types	/IWWRK/T_TGW_LOG_ENTRY_EXPAND		0	0
POTENTIALOWNERS	1 Types	/IWWRK/T_TGW_POTENT_OWNER_EXPN		0	0
COMMENTS	1 Types	/IWWRK/T_TGW_COMMENTS_EXPAND		0	0
ATTACHMENTS	1 Types	/IWWRK/T_TGW_ATTACHMENTS_EXPAND		0	0
CUSTOMATTRIBUTEDATA	1 Types	/IWWRK/T_TGW_CUSTM_ATTR_EXPAND		0	0
TASKDEFINITIONDATA	1 Types	/IWWRK/S_TGW_TASK_DEF_EXPAND		0	0
POTENTIALOWNERSDETAILS	1 Types	/IWWRK/T_TGW_USER_DET_EXPAND		0	0
POSSIBLEAGENTSDETAILS	1 Types	/IWWRK/T_TGW_USER_DET_EXPAND		0	0
SCAITEMS	1 Types	ZCARTAPP_T_SCAITEM		0	0
SCAAPPROVERSHEADER	1 Types	ZCARTAPP_T_SCAAPPROVER		0	0

These structures are used in the internal representation of the expanded Task entity. For each custom entity type and entity set, there must be a DDIC object available. Each field inside a structure must correspond to the OData representation of the properties of the custom entity.

DDIC structures for the ZCARTAPP_ITEMLEVEL_APPROVAL RFC Function Module

Create the DDIC structures below in order to map incoming data to the appropriate internal format.

- ZCARTAPP_S_SCAITEM_APPROVAL

Component	Typing	Component Type	Data Type	Length	Decimal
ITEM_NUMBER	1 Types	CRMT_ITEM_NO	NUMC	10	0

Component	Typing	Component Type	Data Type	Length	Decimal
ITEM_APPROVED	1 Types		CHAR	1	0
WORKITEM_ID	1 Types	SWW_WIID	NUMC	12	0

- ZCARTAPP_T_SCAITEM_APPROVAL
Line Type: ZCARTAPP_S_SCAITEM_APPROVAL
- ZCARTAPP_S_SCAHEADER_APPROVAL

Component	Typing	Component Type	Data Type	Length	Decimal
WORKITEM_ID	1 Types	SWW_WIID	NUMC	12	0
SC_NUMBER	1 Types	CRMT_OBJECT_ID_DB	CHAR	10	0
APPROVAL_NOTE	1 Types		STRING	0	0
SCAAPPROVALITEMS	1 Types	ZCARTAPP_T_SCAITEM_APPROVAL		0	0

Any other DDIC structure that is required for extensibility is generated by the Service Builder.

4 Extending in the Gateway

To display custom fields and enable custom workflows in the My Inbox application, you need to extend the Task Gateway Service, which will result in a new service. Then you need to extend the service implementation and assign a provider to the data model.

4.1 Extending the Task Gateway Service

You can extend the Task Gateway Service using the Service Builder tool (transaction **SEGW**). It generates metadata and runtime classes, and provides the means of extending the source code.

For more information about the Service Builder, see the product documentation at

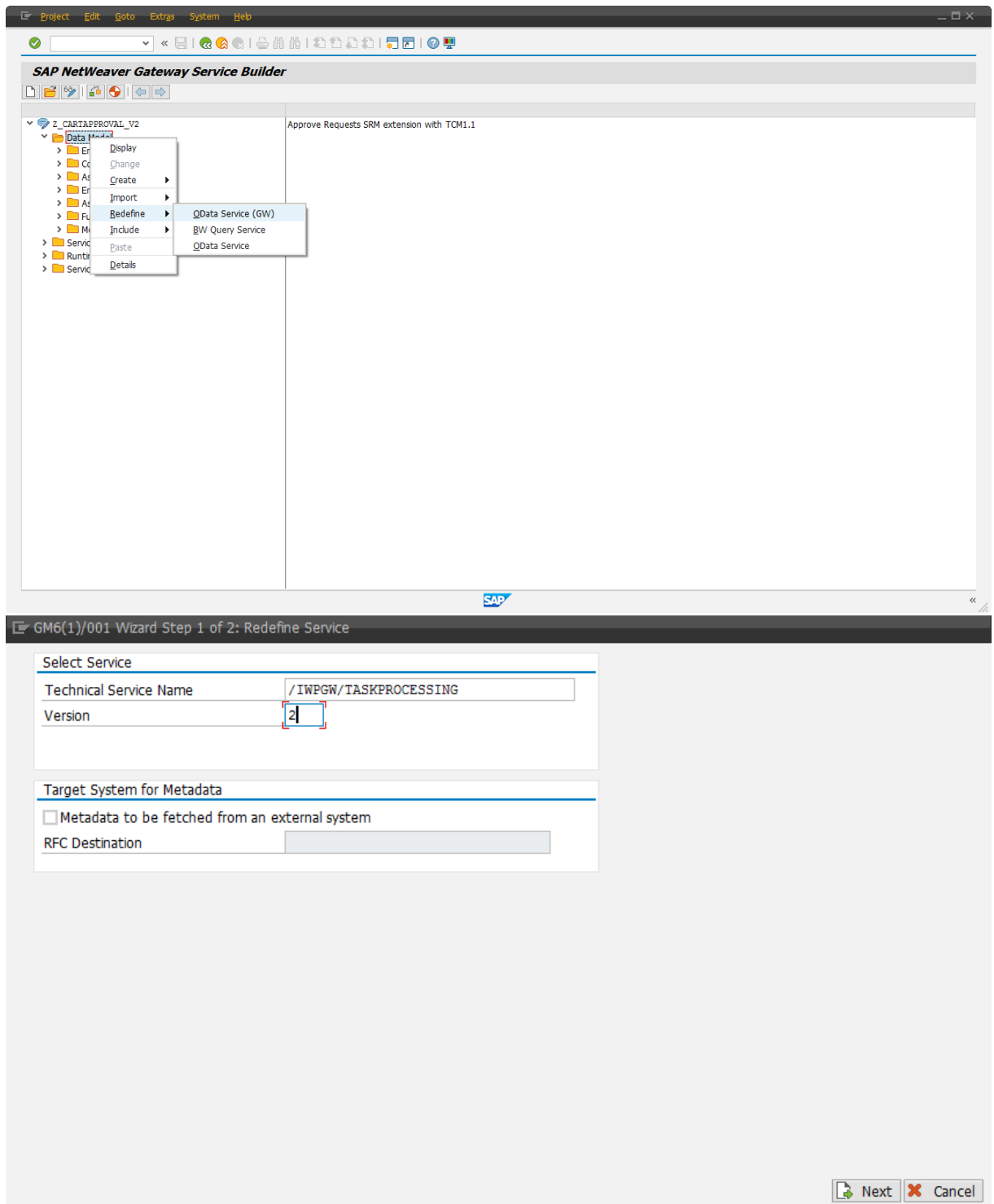
http://help.sap.com/saphelp_gateway20sp09/helpdata/en/1b/c16e1e20a74746ad386bc10b60b6c3/content.htm?frameset=/en/29/801ec804854ee58c805f50b80480cf/frameset.htm¤t_toc=/en/57/a41787789c4eca867d9a09696fc42c/plain.htm&node_id=419&show_children=false.

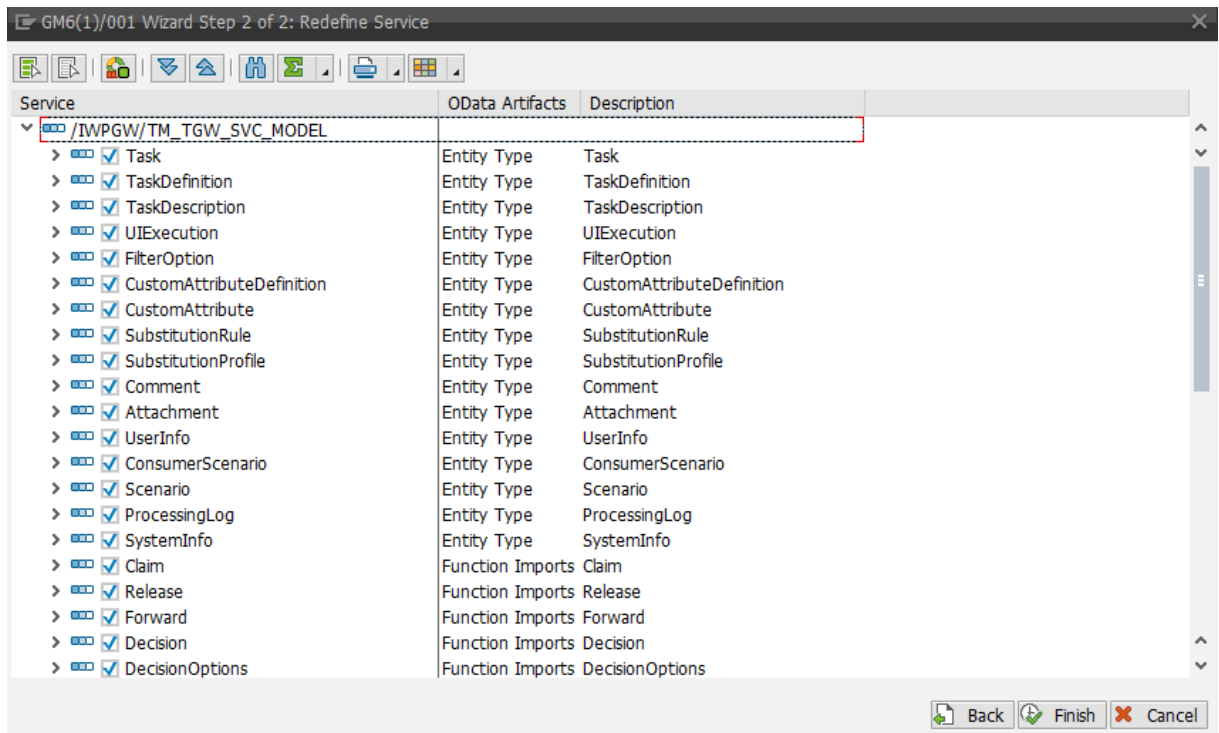
4.1.1 Create a New Service

Create a new service by redefining an existing service.

1. Start transaction **SEGW**.
2. Create a new project.
3. In the empty project, redefine the `/IWPGW/TASKPROCESSING 0002` service.

Select all entities to be reused.





4. Save your project and choose [Generate Runtime Objects](#).

This generates the required implementation and creates a new service. You can check the generated objects later in the `Runtime Artifacts` folder.

Note

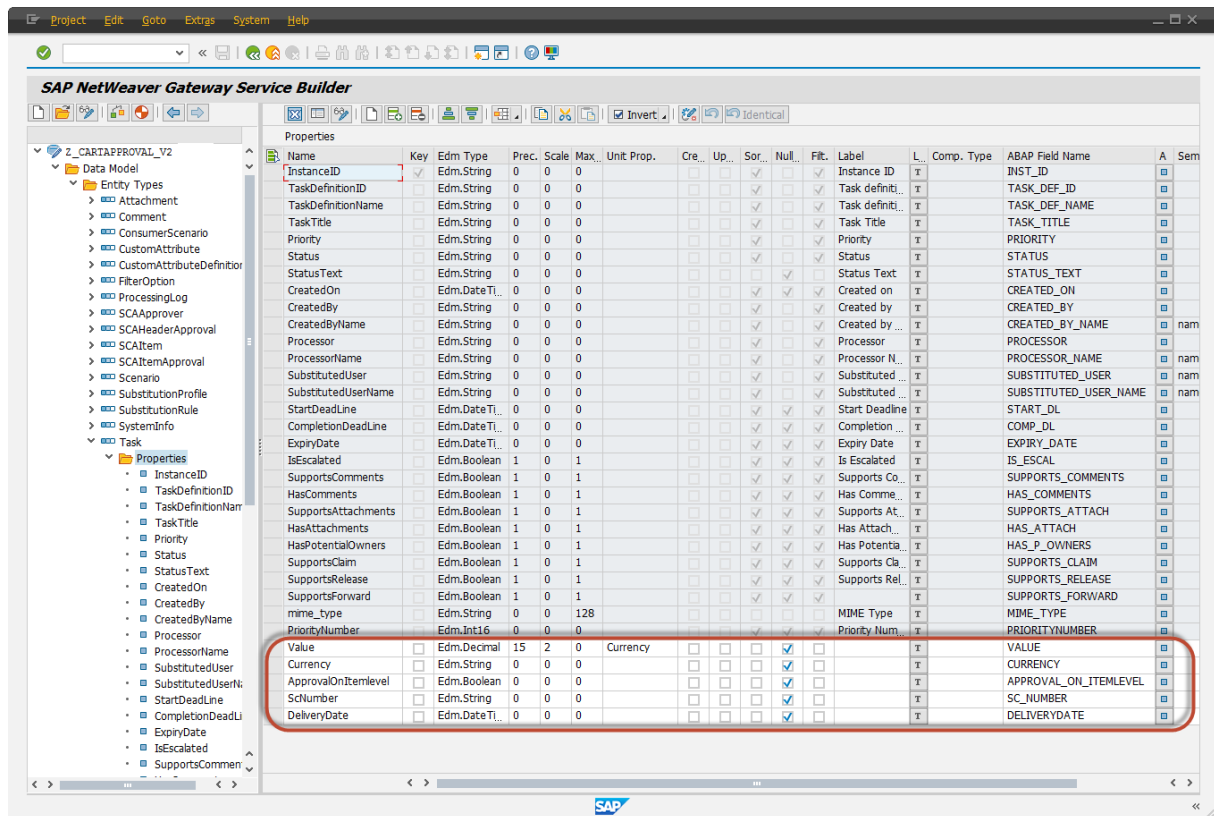
Change the technical service name after you have chosen [Generate Runtime Objects](#).

4.1.2 Add Custom Properties to the Service

The list screen (S2) and the detail screen (S3) are populated with data using the existing task collection. Extend the Task entity with custom properties in order to extend the S2 and S3 screens with custom data.

To add new properties, follow the steps below.

1. In the Service Builder tool, choose [Data Model](#) -> [Entity Types](#) -> [Task](#) and double-click [Properties](#).
2. In the [Properties](#) screen area, add the new fields and enter the details for each of them as shown below.



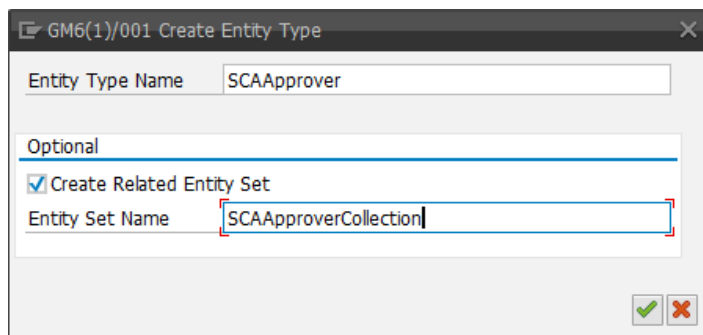
4.1.3 Add the Chain of Approvers to Service

Create a new collection and link it to the Task entity via association and navigation in order to display the chain of approvers on the detail screen.

1. In the Service Builder tool, create a new entity type and a related entity set.

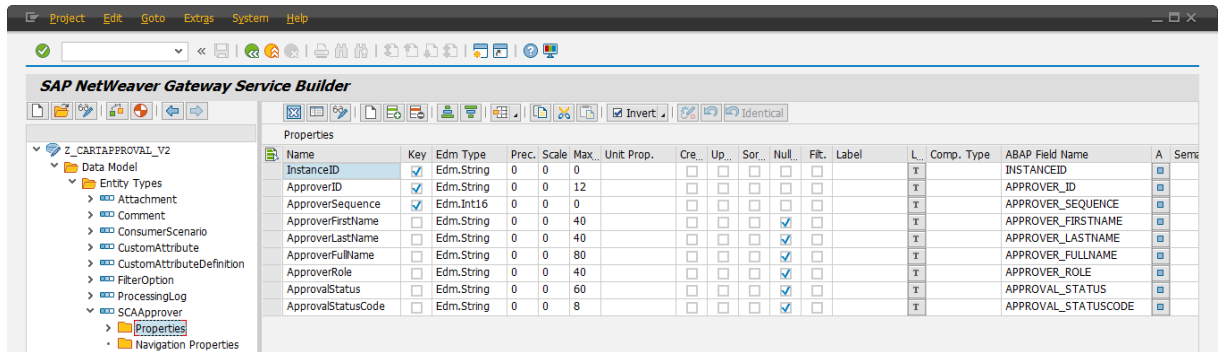
Right-click **Entity Types**, then choose **Create**.

2. Set its keys and properties.



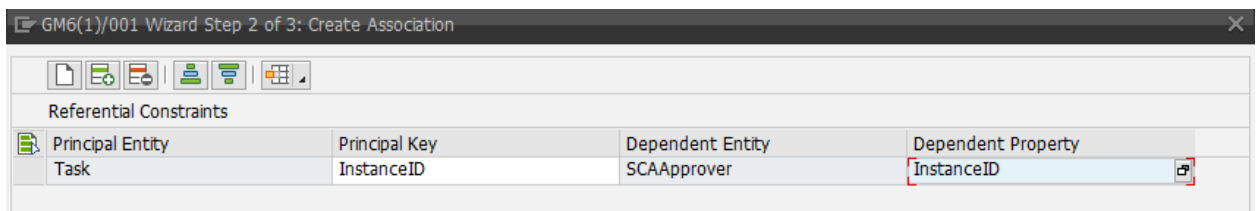
1 Note

The InstanceID property is the instance ID of the principal entity (task), and it is required for OData navigation.

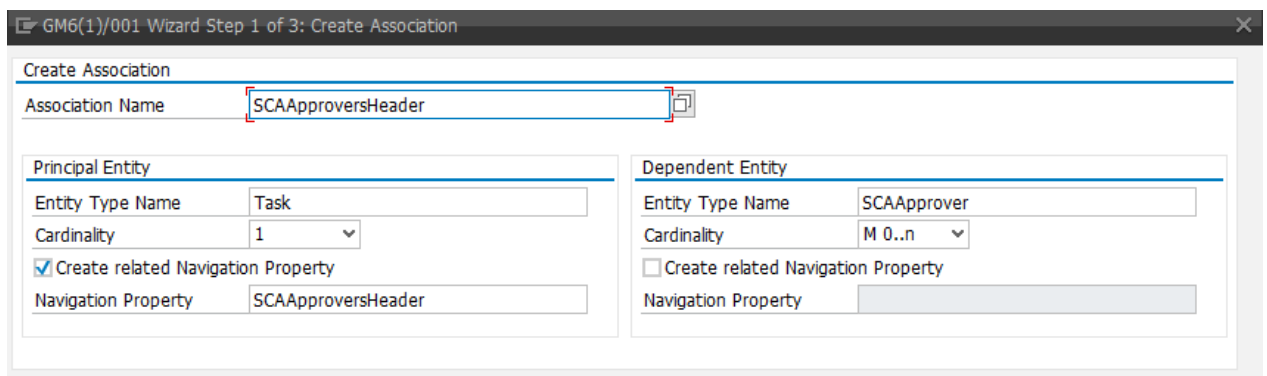


3. Create an association between the new entity type and the Task entity type.

Go to **Data Model -> Associations**, and in the context menu choose **Create**. Enter the details, and create a related



navigation property from the principal entity as shown below.



4.1.4 Add the Shopping Cart Line Item List to the Service

Create a new collection and link it to the Task entity via association and navigation in order to display the line item list (S3) and details (S4) in the application.

1. In the Service Builder tool, create a new entity type and a related entity set.

Right-click **Entity Types**, then choose **Create**.

GM6(1)/001 Create Entity Type

Entity Type Name: SCAItem

Optional

☒ Create Related Entity Set

Entity Set Name: SCAItemCollection

☐ ☐

- Set its keys and properties.

1 Note

The InstanceID property is the instance ID of the principal entity (task), and it is required for OData navigation.

SAP NetWeaver Gateway Service Builder

Properties

Name	Key	Edm Type	Prec.	Scale	Max..	Unit	Prop.	Cre..	Up..	Sor..	Null..	Fit.	Label	L..	Comp.	Type	ABAP Field Name	A	Sem
InstanceID	☒	Edm.String	0	0	0			☐	☐	☐	☐	☐					INSTANCE_ID	☐	
ItemNumber	☒	Edm.String	0	0	10			☐	☐	☐	☐	☐					ITEM_NUMBER	☐	
Description	☐	Edm.String	0	0	40			☐	☐	☐	☒	☐					DESCRIPTION	☐	
VendorId	☐	Edm.String	0	0	10			☐	☐	☐	☒	☐					VENDOR_ID	☐	
VendorName	☐	Edm.String	0	0	40			☐	☐	☐	☒	☐					VENDOR_NAME	☐	
Unit	☐	Edm.String	0	0	3			☐	☐	☐	☒	☐					UNIT	☐	
UnitText	☐	Edm.String	0	0	10			☐	☐	☐	☒	☐					UNIT_TEXT	☐	
Quantity	☐	Edm.String	0	0	0			☐	☐	☐	☒	☐					QUANTITY	☐	
Value	☐	Edm.Decimal	15	2	0	Currency		☐	☐	☐	☒	☐					VALUE	☐	
Currency	☐	Edm.String	0	0	5			☐	☐	☐	☒	☐					CURRENCY	☐	
Price	☐	Edm.Decimal	13	2	0	PriceUnit		☐	☐	☐	☒	☐					PRICE	☐	
PriceUnit	☐	Edm.Decimal	0	0	0			☐	☐	☐	☒	☐					PRICE_UNIT	☐	
DeliveryDate	☐	Edm.DateTime	0	0	0			☐	☐	☐	☒	☐					DELIVERY_DATE	☐	
CategoryDescription	☐	Edm.String	0	0	40			☐	☐	☐	☒	☐					CATEGORY_DESCRIPTION	☐	

- Create an association between the new entity and the Task entity.

Go to [Data Model -> Association](#), and in the context menu choose [Create](#). Enter the details, and create a related navigation property from the principal entity as shown below.

Create Association

Association Name SCAItems

Principal Entity		Dependent Entity	
Entity Type Name	Task	Entity Type Name	SCAItem
Cardinality	1	Cardinality	M 0..n
☒ Create related Navigation Property		☐ Create related Navigation Property	
Navigation Property	SCAItems	Navigation Property	

GM6(1)/001 Wizard Step 2 of 3: Create Association

Referential Constraints

Principal Entity	Principal Key	Dependent Entity	Dependent Property
Task	InstanceID	SCAItem	InstanceID

4.1.5 Enable Item-Level Approval

In this example (shopping cart) scenario, we extend the standard app, so that it is capable of both header-level and item-level approval.

Create custom entities and collections that are used for item-level approval only. The client can initiate a create operation (POST) on this entity and the item collection, which is interpreted as a decision on item-level by the back end.

There are two entities required: one for the header level to identify the Shopping Cart, and one for item-level approval. You need to create an association and navigation between the two entities. The client can execute a deep insert operation on this hierarchy, that is, it can send a decision for every line item.

1. Create a new entity and a related entity set for the header level.

Entity Type Name SCAHeaderApproval

Optional

☒ Create Related Entity Set

Entity Set Name SCAHeaderApprovalCollection

Properties

Name	Key	Edm Type	Prec.	Scale	Max.	Unit	Prop.	Cre...	Up...	Sor...	Null...	Flt.	Label	L...	Comp.	Type	ABAP Field Name	A	Sem...
WorkitemID	☒	Edm.String	0	0	0			☐	☐	☐	☐	☐					WORKITEM_ID	☐	
SCNumber	☐	Edm.String	0	0	0			☐	☐	☐	☐	☐					SC_NUMBER	☐	
Comment	☐	Edm.String	0	0	0			☐	☐	☐	☐	☐					APPROVAL_NOTE	☐	

2. Create a new entity and a related entity set for the item level.

Entity Type Name

Optional

☒ Create Related Entity Set

Entity Set Name

SAP NetWeaver Gateway Service Builder

Properties

Name	Key	Edm Type	Prec.	Scale	Max...	Unit Prop.	Cre...	Up...	Sor...	Null...	Flt.	Label	L...	Comp.	Type	ABAP Field Name	A	Sema
WorkitemID	☒	Edm.String	0	0	0		☐	☐	☐	☐	☐					WORKITEM_ID	☐	
ItemNumber	☒	Edm.String	0	0	0		☐	☐	☐	☐	☐					ITEM_NUMBER	☐	
ItemApproved	☐	Edm.String	0	0	0		☐	☐	☐	☐	☐					ITEM_APPROVED	☐	

3. Create an association and navigation between the two entities.

Create Association

Association Name

Principal Entity

Entity Type Name

Cardinality

☒ Create related Navigation Property

Navigation Property

Dependent Entity

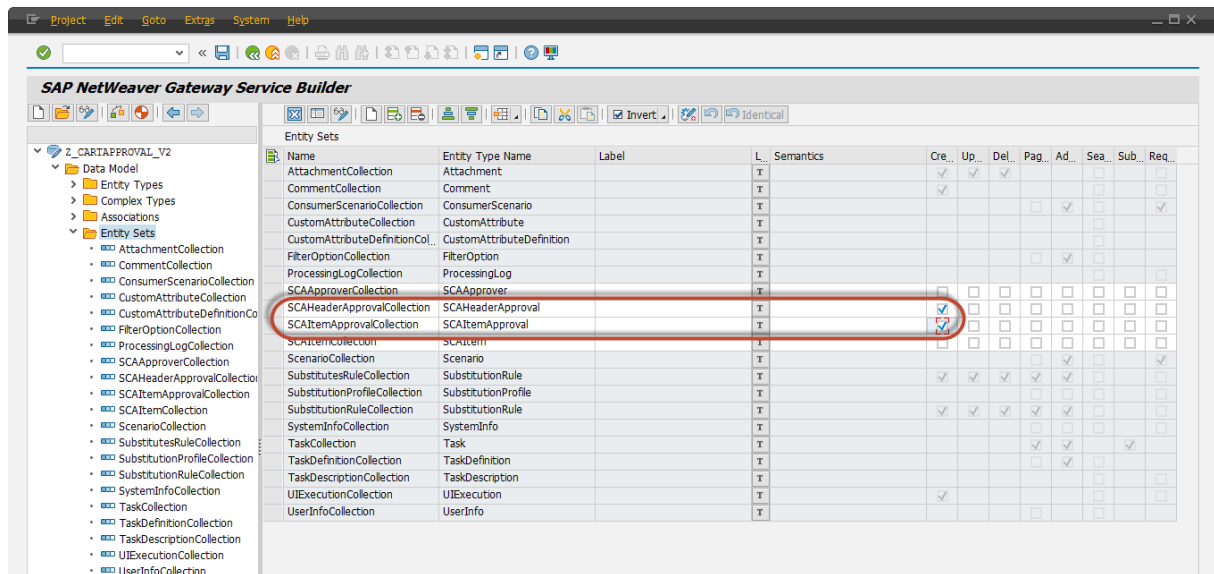
Entity Type Name

Cardinality

☐ Create related Navigation Property

Navigation Property

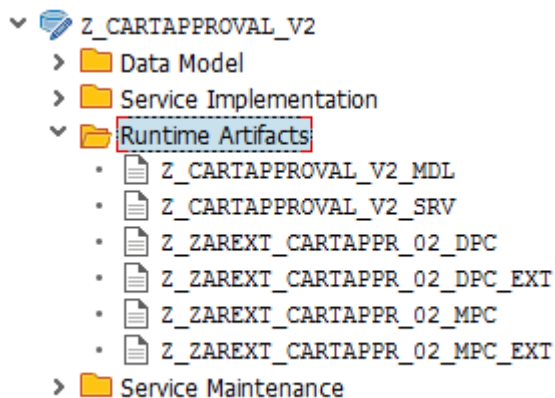
4. Set the new entity sets as creatable.



5. Save and generate runtime objects.

4.2 Extending the Service Implementation

The Service Builder generates runtime (DPC) and metadata (MPC) classes for the new service, and creates a subclass for each with the suffix `_EXT`. These subclasses contain those methods that you need to redefine with your custom implementation. You can always check the class name under the [Runtime Artifacts](#) node.



The client application uses the following queries against the back end to retrieve business data (including custom entities) for tasks, and to execute decisions:

- Task list (S2)

```
GET TaskCollection?$skip=0&$top=100&$orderby=CreatedOn
desc&$inlinecount=allpages&$filter=(SAP__Origin eq '<SAP System Alias>' and
(TaskDefinitionID eq '<Custom Task Def. ID>') and (Status eq 'READY' or Status
eq 'RESERVED' or Status eq 'IN_PROGRESS' or Status eq 'EXECUTED'))
```
- Task details (S3) including tabs

```
GET TaskCollection(SAP__Origin='<SAP System Alias>',InstanceID='<Custom Task
ID>'?$expand=Attachments,Description,CustomAttributeData,UIExecutionLink,Comment
s,SCAItems,SCAApproversHeader
```


- Item details (S4)

```
GET SCAItemCollection(SAP__Origin='<SAP System Alias>',InstanceID='<Custom Task ID > ',ItemNumber='<Custom Item ID>')
```
- Item-level decision execution

```
POST SCAHeaderApprovalCollection
{
  "WorkitemID":"<Custom Task ID>",
  "ScNumber":"<Custom Process ID>",
  "Comment":"",
  "SCAApprovalItems":[
    {
      "WorkitemID":"<Custom Task ID>",
      "ItemNumber":"<Custom Item ID 1>",
      "ItemApproved":"X"
    },
    {
      "WorkitemID":"<Custom Task ID>",
      "ItemNumber":"<Custom Item ID 2>",
      "ItemApproved":"X"
    },
    ...
  ]
}
```

Each query is mapped to one method of the DPC class. Redefine these methods with custom implementation as described in the sections below.

Note

There are other options to override the query implementations. The examples below are recommendations that you can use in similar extensibility scenarios

4.2.1 Retrieve Business Data for the Task List (S2 Screen)

The client application retrieves the Task list by sending a `GET TaskCollection` command to the Gateway.

Open the runtime `_EXT` class and redefine the `ENTITYSET_TASK` method in order to fill custom fields with values. The implementation retrieves the details of the workflow tasks, and invokes the remote back end for additional data. Task details and additional data are mapped to the extended task structure.

Enter the following source code in the `_EXT` class:

```
METHOD entityset_task.

DATA:
  lo_tasks          TYPE REF TO /iwpgw/if_tgw_types=>tt_tasks,
  ls_task           TYPE /iwpgw/if_tgw_types=>ty_task,
  lt_tasks_ext      TYPE zcl_z_cartapprova_mpc_ext=>tt_task,
  ls_task_ext       TYPE zcl_z_cartapprova_mpc_ext=>ts_task,
*  structures for RFC
  lt_rfc_tasklist   TYPE zcartapp_t_apv_sc_wf_list,
  ls_rfc_task       TYPE zcartapp_s_apv_sc_wf_list.
```

```

* get work item details
CALL METHOD super->entityset_task
  RECEIVING
    rr_entityset = rr_entityset.

* call backend for custom field values
CALL FUNCTION 'ZCARTAPP_GET_WFTASKLIST' DESTINATION 'S3FCLNT300_T'
  IMPORTING
    et_list_data = lt_rfc_tasklist.

* map results
CREATE DATA lo_tasks.
lo_tasks ?= rr_entityset.
LOOP AT lo_tasks->* INTO ls_task.
  CLEAR ls_task_ext.
  MOVE-CORRESPONDING ls_task TO ls_task_ext.
  READ TABLE lt_rfc_tasklist INTO ls_rfc_task WITH KEY workitem_id = ls_task-
inst_id.
  ls_task_ext-value = ls_rfc_task-value.
  ls_task_ext-currency = ls_rfc_task-currency.
  ls_task_ext-approval_on_itemlevel = ls_rfc_task-approval_on_itemlevel.
  ls_task_ext-sc_number = ls_rfc_task-sc_number.
  APPEND ls_task_ext TO lt_tasks_ext.
ENDLOOP.

CALL METHOD copy_data_to_ref
  EXPORTING
    is_data = lt_tasks_ext
  CHANGING
    cr_data = rr_entityset.

ENDMETHOD.

```

Note

Change the RFC destination S3FCLNT300_T according to your configuration settings.

4.2.2 Retrieve Business Data for the Task Details, Task Items, and Approver Chain (S3 Screen)

The client application retrieves the Task details (including comments, attachments, and custom entities.) by sending a GET Task command to the Gateway using \$expand with the required details.

Open the runtime `_EXT` class and redefine the `CASE_EXPANDED_ENTITY` method in order to fill custom fields with values. The implementation retrieves all required details from the remote back end and maps them to the used structures. When the Task is not queried, the original implementation is executed.

Enter the following source code in the `_EXT` class:

```

METHOD case_expanded_entity.

  DATA:
    lr_expanded_task_ext TYPE REF TO zcartapp_s_tgw_expand_deep,
    lr_exp_task          TYPE REF TO /iwpgw/wf_tgw_expand=>/iwwrk/s_tgw_expand
    _deep,

```

```

        lr_entity                TYPE REF TO data,
        ls_scaitem               TYPE zcartapp_s_scaitem,
        lt_scaitem               TYPE zcartapp_t_scaitem,
        ls_scaapprover           TYPE zcartapp_s_scaapprover,
        lt_scaapprover           TYPE zcartapp_t_scaapprover,
        ls_key                   TYPE /iwbep/s_mgw_name_value_pair,
        ls_key_item              TYPE /iwbep/s_mgw_name_value_pair,
*
        variables for RFC
        lv_wiid                  TYPE sww_wiid,
        ls_header                 TYPE zcartapp_s_apv_sc_hd,
        ls_item                   TYPE zcartapp_s_apv_sc_it,
        lt_item                   TYPE zcartapp_t_apv_sc_it,
        ls_approver               TYPE zcartapp_s_apv_sc_approver,
        lt_approver               TYPE zcartapp_t_apv_sc_approver.

* get work item details
CALL METHOD super->case_expanded_entity
EXPORTING
    iv_entity_name               = iv_entity_name
    iv_entity_set_name           = iv_entity_set_name
    iv_source_name               = iv_source_name
    it_key_tab                    = it_key_tab
    it_navigation_path           = it_navigation_path
    io_expand                     = io_expand
    io_tech_request_context       = io_tech_request_context
    it_req_expand                 = it_req_expand
IMPORTING
    er_entity                    = lr_entity
    et_expanded_clauses           = et_expanded_clauses
    et_expanded_tech_clauses      = et_expanded_tech_clauses.

* call backend for custom field values
READ TABLE it_key_tab INTO ls_key WITH KEY name = 'InstanceID'.
READ TABLE it_key_tab INTO ls_key_item WITH KEY name = 'ItemNumber'.
lv_wiid = ls_key-value.
CALL FUNCTION 'ZCARTAPP_GET_SC_DETAILS' DESTINATION 'S3FCLNT300_T'
EXPORTING
    iv_workitem_id = lv_wiid
IMPORTING
    es_header       = ls_header
    et_item          = lt_item
    et_approver      = lt_approver.

* map results
CREATE DATA lr_expanded_task_ext.
lr_exp_task ?= lr_entity.
MOVE-CORRESPONDING lr_exp_task->* TO lr_expanded_task_ext->*.

lr_expanded_task_ext->value = ls_header-value.
lr_expanded_task_ext->currency = ls_header-currency.
lr_expanded_task_ext->approval_on_itemlevel = ls_header-approval_on_itemlevel.
lr_expanded_task_ext->sc_number = ls_header-sc_number.

LOOP AT lt_item INTO ls_item.
    ls_scaitem-instance_id = lv_wiid.
    ls_scaitem-item_number = ls_item-item_number.
    ls_scaitem-description = ls_item-description.
    ls_scaitem-vendor_id   = ls_item-vendor_id.
    ls_scaitem-vendor_name = ls_item-vendor_name.

```

```

ls_scaitem-unit          = ls_item-unit.
ls_scaitem-unit_text    = ls_item-unit_text.
ls_scaitem-quantity     = ls_item-quantity.
ls_scaitem-value        = ls_item-value.
ls_scaitem-currency     = ls_item-currency.
ls_scaitem-price        = ls_item-price.
ls_scaitem-price_unit   = ls_item-price_unit.
ls_scaitem-delivery_date = ls_item-delivery_date.
ls_scaitem-category_description = ls_item-category_description.
APPEND ls_scaitem TO lt_scaitem.
ENDLOOP.
lr_expanded_task_ext->scaitems = lt_scaitem.

LOOP AT lt_approver INTO ls_approver.
  ls_scaapprover-instance_id      = lv_wiid.
  ls_scaapprover-approver_id      = ls_approver-approver_id.
  ls_scaapprover-approver_sequence = ls_approver-approver_sequence.
  ls_scaapprover-approver_firstname = ls_approver-approver_firstname.
  ls_scaapprover-approver_lastname = ls_approver-approver_lastname.
  ls_scaapprover-approver_fullname = ls_approver-approver_fullname.
  ls_scaapprover-approver_role     = ls_approver-approver_role.
  ls_scaapprover-approval_status  = ls_approver-approval_status.
  ls_scaapprover-approval_statuscode = ls_approver-approval_statuscode.
  APPEND ls_scaapprover TO lt_scaapprover.
ENDLOOP.
lr_expanded_task_ext->scaapproversheader = lt_scaapprover.

er_entity = lr_expanded_task_ext.

ENDMETHOD.

```

Note

Change the RFC destination S3FCLNT300_T according to your configuration settings.

4.2.3 Retrieve Business Data for Item Details (S4 Screen)

The client application retrieves the item details by sending a GET SCAItem command to the Gateway.

Open the runtime `_EXT` class, and redefine the `/IWBEF/IF_MGW_APPL_SRV_RUNTIME~GET_ENTITY` method in order to fill the item details with values. The implementation retrieves all required details from the remote back end and maps them to the used structures. When the Task is not queried, the original implementation is executed.

Enter the following source code in the `_EXT` class:

```

METHOD /iwbef/if_mgw_appl_srv_runtime~get_entity.

  DATA:
    ls_scaitem      TYPE zcl_z_cartapproval_mpc=>ts_scaitem,
    ls_scaapprover  TYPE zcl_z_cartapproval_mpc=>ts_scaapprover,
    ls_key          TYPE /iwbef/s_mgw_name_value_pair,
    ls_key_item     TYPE /iwbef/s_mgw_name_value_pair,
    * RFC structures

    lv_wiid         TYPE sww_wiid,

```

```

ls_header      TYPE zcartapp_s_apv_sc_hd,
lt_item        TYPE zcartapp_t_apv_sc_it,
lt_approver    TYPE zcartapp_t_apv_sc_approver,
ls_item        TYPE zcartapp_s_apv_sc_it,
ls_approver    TYPE zcartapp_s_apv_sc_approver.

CASE iv_entity_name.
  WHEN 'SCAitem'.
*    get Shopping Cart key field values
    READ TABLE it_key_tab INTO ls_key WITH KEY name = 'InstanceID'.
    READ TABLE it_key_tab INTO ls_key_item WITH KEY name = 'ItemNumber'.
    lv_wiid = ls_key-value.

*    retrieve Shopping Cart details
    CALL FUNCTION 'ZCARTAPP_GET_SC_DETAILS' DESTINATION 'S3FCLNT300_T'
      EXPORTING
        iv_workitem_id = lv_wiid
      IMPORTING
        es_header      = ls_header
        et_item        = lt_item
        et_approver    = lt_approver.

*    map details
    READ TABLE lt_item INTO ls_item WITH KEY item_number = ls_key_item-value.
    ls_scaitem-instance_id = lv_wiid.
    ls_scaitem-item_number = ls_item-item_number.
    ls_scaitem-description = ls_item-description.
    ls_scaitem-vendor_id   = ls_item-vendor_id.
    ls_scaitem-vendor_name = ls_item-vendor_name.
    ls_scaitem-unit        = ls_item-unit.
    ls_scaitem-unit_text   = ls_item-unit_text.
    ls_scaitem-quantity    = ls_item-quantity.
    ls_scaitem-value       = ls_item-value.
    ls_scaitem-currency    = ls_item-currency.
    ls_scaitem-price       = ls_item-price.
    ls_scaitem-price_unit  = ls_item-price_unit.
    ls_scaitem-delivery_date = ls_item-delivery_date.
    ls_scaitem-category_description = ls_item-category_description.
    CALL METHOD copy_data_to_ref
      EXPORTING
        is_data = ls_scaitem
      CHANGING
        cr_data = er_entity.

  WHEN OTHERS.

    CALL METHOD super->/iwbeb/if_mgw_appl_srv_runtime~get_entity
      EXPORTING
        iv_entity_name      = iv_entity_name
        iv_entity_set_name   = iv_entity_set_name
        iv_source_name       = iv_source_name
        it_key_tab           = it_key_tab
        it_navigation_path    = it_navigation_path
        io_tech_request_context = io_tech_request_context
      IMPORTING
        er_entity           = er_entity.

ENDCASE.

```

ENDMETHOD.

Note

Change the RFC destination S3FCLNT300_T according to your configuration settings.

4.2.4 Enable Item-Level Approval

To enable item-level approval use a deep insert on a dedicated entity type hierarchy (header-item).

Open the runtime `_EXT` class, and redefine the `/IWBEP/IF_MGW_APPL_SRV_RUNTIME~CREATE_DEEP_ENTITY` method. The implementation maps the incoming decision data into a format, which the remote back end understands, and sends a request to the back end.

Enter the following source code in the `_EXT` class:

```
METHOD /iwbsp/if_mgw_appl_srv_runtime~create_deep_entity.

    DATA: ls_header_approval TYPE zcartapp_s_scaheader_approval,
           lt_item_decision   TYPE zcartapp_t_scaitem_approval.

    DATA: lv_workitem_id      TYPE swi_wiid,
           lv_approval_note    TYPE string.

    CASE iv_entity_name.

        WHEN 'SCAHeaderApproval'.

            io_data_provider->read_entry_data( IMPORTING es_data = ls_header_approval ).

            lv_workitem_id      = ls_header_approval-workitem_id.
            lv_approval_note    = ls_header_approval-approval_note.
            lt_item_decision    = ls_header_approval-scaapprovalitems.

            CALL FUNCTION 'ZCARTAPP_ITEMLEVEL_APPROVAL' DESTINATION 'S3FCLNT300_T'
                EXPORTING
                    iv_work_item      = lv_workitem_id
                    iv_approval_note  = lv_approval_note
                    it_item_decision   = lt_item_decision.

            IF sy-subrc EQ 0.
                copy_data_to_ref( EXPORTING is_data = ls_header_approval
                                CHANGING cr_data = er_deep_entity ).
            ELSE.
                RAISE EXCEPTION TYPE /iwbsp/cx_mgw_busi_exception.
            ENDIF.

    WHEN OTHERS.
        RETURN.
```

ENDCASE.

ENDMETHOD.

Note

For more information about the payload format, see section [Extending the Service Implementation](#)

4.3 Configuring the Service in the Gateway

Make sure that you have completed the following customizing activities:

- Configuring a Task and Decisions (back-end system)
- Defining the Workflow Scenario (Gateway)

For more information, see the My Inbox product documentation on SAP Help Portal at <http://help.sap.com/fiori>.

4.3.1 Add and Configure the Customer Service

Register and activate the extended service in the Gateway before consumption.

1. Start transaction `/IWFND/MAINT_SERVICE`.
2. Choose [Add Service](#).
3. Select `LOCAL` as your system alias, and add the custom service.
4. Add system aliases to your service.

For more information, see

http://help.sap.com/saphelp_gateway20sp09/helpdata/en/29/801ec804854ee58c805f50b80480cf/frameset.htm.

4.3.2 Assign Provider to Data Model

Since you have created a custom service that acts as the extended Task Gateway Service, you need to assign the `/IWFND/BWF` data provider to the new service.

1. In Customizing for SAP NetWeaver, choose [Gateway -> OData Channel Development with IW_BEP -> Registration](#), and open [Assign Data Provider to Data Model](#)

Note

If you run SAP NetWeaver 7.40, choose [Gateway -> OData Channel -> Administration -> General Settings](#) and open [Assign Data Provider to Data Model](#).

2. Create a new entry as described below:
 - Model identifier: `<generated model name>_0001_BE` (you can find this in Service Builder, under [Generated Artifact](#) as [Registered Model](#))

- Type: **Adaptation (Data Provider)**
- Software Version: **/IWFND/BWF**
- Class/Interface: **/IWFND/CL_MGW_RUNT_RCLNT_PRXY**

3. Save your entry.

4.4 Troubleshooting in the Gateway System

Symptom

There is no ABAP structure binding for the extended entity types.

Reason

The Task entity does not have a bound structure.

Solution

Bind the structure manually in the metadata `_EXT` class as described below:

1. Open the metadata `_EXT` class.
2. Redefine the `DEFINE` method, and insert the following source code:

```
METHOD define.
```

```
  DATA:
```

```
    lo_entity_type      TYPE REF TO /iwbep/if_mgw_odata_entity_typ.
```

```
  CALL METHOD super->define( ).
```

```
    lo_entity_type = model->get_entity_type( iv_entity_name = 'Task' ).
```

```
    lo_entity_type-
```

```
>bind_structure( iv_structure_name = 'ZCL_Z_CARTAPPROVAL_MPC=>TS_TASK' ).
```

```
ENDMETHOD.
```


5 Extending the Front End (UI5 Code Extension)

After you have extended the Task Gateway Service and the service implementation, you need to perform the extensibility tasks described below in your front-end system.

5.1 Create an Extended App Skeleton

As a first step create a new SAPUI5 application project in Eclipse and upload it to the same system where your standard application resides.

In this example, we use the following JavaScript namespace: `cross.fnd.fiori.inbox_ext`

You can create a new project manually in Eclipse.

5.1.1 Create a New Project

Create a new project manually as described below.

1. Open Eclipse and choose [New... > Other... > SAPUI5 Application Development > Application Project](#).
2. Enter a name (for example, `cross.fnd.fiori.inbox_ext`) and select [Mobile](#) as [Target Device](#).
3. Deselect the [Create an Initial View](#) checkbox.
4. Choose [Finish](#).
5. In the WebContent folder of the project, create a new file with the name `Component.js`.

This file will be the entry point of the extended application when it is running in Fiori Launchpad.

1. Open the file and declare the `Component` in the first line using the following naming convention:

```
<namespace>.Component
```

```
Enter jQuery.sap.declare("cross.fnd.fiori.inbox_ext.Component");
```

2. In the next line, register the module path of the standard application.

The first parameter is the namespace of the standard application (`cross.fnd.fiori.inbox`) and the second one is the location path of the extended application.

```
Enter jQuery.sap.registerModulePath("cross.fnd.fiori.inbox",  
'/sap/bc/ui5_ui5/sap/ca_fiori_inbox');
```

The `Component.js` of the extended application is inherited from the standard application. The component has a metadata property which defines the configuration, including the custom OData service, customizing with the extension point definitions, and the routing that defines the navigation paths to the custom views.

In this example, the structure of the `Components.js` is the following:

```

jQuery.sap.declare("cross.fnd.fiori.inbox_ext.Component");

jQuery.sap.registerModulePath("cross.fnd.fiori.inbox",
    '/sap/bc/ui5_ui5/sap/ca_fiori_inbox');

jQuery.sap.require("cross.fnd.fiori.inbox.Component");

cross.fnd.fiori.inbox.Component.extend("cross.fnd.fiori.inbox_ext.Component", {

    metadata: {

        version : "1.0",

        config : {

            "sap.ca.serviceConfigs":

            [{

                name: "TASKPROCESSING",

                masterCollection: "TaskCollection",

                serviceUrl: "/sap/opu/odata/sap/Z_CARTAPPROVAL_V2_SRV;mo",

                isDefault: true,

            },

            {

                name: "POSTACTION",

                masterCollection: "TaskCollection",

                serviceUrl: "/sap/opu/odata/sap/Z_CARTAPPROVAL_V2_SRV;mo",

                isDefault: false,

                useBatch: true

            }

        ]

    },

    customizing: {

    }

}

});

```

For My Inbox extensions, it is required to create these two `serviceConfigs` objects with the custom service URL.

5.1.2 Upload the New Project

Upload the application skeleton to the same system where your standard application resides as a BSP application.

You can upload the app skeleton manually or by using the Team Provider wizard, which is a tool available in SAPUI5 Eclipse by default. Team Provider enables you to submit and retrieve code changes regularly and to resolve code change conflicts in a distributed development.

We suggest that you upload the app skeleton with Team Provider

1. Right-click the custom project in Eclipse and choose [Share project...](#) from the context menu.
2. Select [SAPUI5 ABAP Repository](#).
3. Specify the target system, your user name, and password.

You can also use single-sign on.

4. On the next screen, specify the BSP repository.

You can define a new BSP application or choose from applications already available.

After completing the steps above, the project is shared and you can submit the local files from the project.

5. Right-click the project and choose [Team > Submit](#) from the context menu.

Make sure that you upload all the required files from the `WebContent` folder.

5.1.3 Running in Standalone Mode

For the development as well as the testing of the extended application, we suggest creating an alternative application entry point that enables the application to run in standalone mode, without the Fiori Launchpad.

To do so, create another file, called `noShellIndex.html`. This name differs from the common naming convention of `index.html` to emphasize its purpose. Update the `web.xml` of the project with the new html file name as shown below:

```
<welcome-file-list>

    <welcome-file>noShellIndex.html</welcome-file>

</welcome-file-list>
```

The `noShellIndex.html` file contains the bootstrap of the SAPUI5 library, the module path registrations of the standard and custom applications and the instantiation of the Component described above including its injection to the DOM of the document.

Create the `noShellIndex.html` file as shown below:

```
<!DOCTYPE HTML>

<html>

    <head>

        <meta http-equiv="X-UA-Compatible" content="IE=edge">

        <meta http-equiv='Content-Type' content='text/html; charset=UTF-8' />
```

```

<script id="sap-ui-bootstrap" type="text/javascript"
        src="/sap/public/bc/ui5_ui5/1-next/resources/sap-ui-
cachebuster/sap-ui-core.js" data-sap-ui-libs="sap.m, sap.me"

        data-sap-ui-theme="sap_bluecrystal"

        data-sap-ui-xx-bindingSyntax="complex"

        data-sap-ui-resourceroots='{ "cross.fnd.fiori.inbox":
"/sap/bc/ui5_ui5/sap/ca_fiori_inbox/" }'>

</script>

<!-- only load the mobile lib "sap.m" and the "sap_mvi" theme -->

<script>

jQuery.sap.require("sap.ui.core.CustomizingConfiguration");

// Registers the application

jQuery.sap.registerModulePath("cross.fnd.fiori.inbox_ext", './');

jQuery.sap.registerModulePath("cross.fnd.fiori.inbox",
'/sap/bc/ui5_ui5/sap/ca_fiori_inbox/');

// Registers SAP UI5

jQuery.sap.registerModulePath("sap.ca.ui", './resources/sap/ca/ui/');

sap.ui.getCore().attachInitEvent(function () {

var oComponentContainer = new sap.ui.core.ComponentContainer({

    height : "100%",

    name:"cross.fnd.fiori.inbox_ext"

});

oComponentContainer.placeAt("content");

});

</script>

</head>

```

```

        <body class="sapUiBody" role="application">

            <div id="content"></div>

        </body>

</html>

```

Running in Local Mode

You can run the extended application in local mode first before uploading it as a BSP application.

To do so, make the following changes in the `noShellIndex.html` file you have just created:

- Replace `src="/sap/public/bc/ui5_ui5/1-next/resources/sap-ui-cachebuster/sap-ui-core.js"` with `src="resources/sap-ui-core.js"` in the bootstrap
- Replace `data-sap-ui-resourceroots='{ "cross.fnd.fiori.inbox": "/sap/bc/ui5_ui5/sap/ca_fiori_inbox/" }'` with `data-sap-ui-resourceroots='{ "cross.fnd.fiori.inbox": "/cross.fnd.fiori.inbox/" }'`
- Comment the line `jQuery.sap.registerModulePath("cross.fnd.fiori.inbox", '/sap/bc/ui5_ui5/sap/ca_fiori_inbox/')`

In the `Component.js` file, comment the line

```

jQuery.sap.registerModulePath("cross.fnd.fiori.inbox",
'/sap/bc/ui5_ui5/sap/ca_fiori_inbox/');

```

5.2 Add New Fields to the List (S2) and Detail (S3) Screens

After creating the extended app skeleton, you can add custom fields to the list and detail screens.

In this example, we add two fields, value and currency, to the list item on the list screen and to the object header on the detail screen.

Use the following extension points to extend the list and detail screens with new fields:

- List screen (`S2.view.xml`): `CustomerExtensionForObjectListItem`
- Detail screen (`S3.view.xml`): `CustomerExtensionForObjectHeader`

Note

Before adding new fields to the extended app skeleton, make sure that you have already extended the Gateway service with custom properties.

5.2.1 Define the View Extensions in the `Component.js`

In the `WebContent` folder of the project, open the `Component.js` file in the extended app and add the following JSON code part under `metadata`, under `customizing`:

```
"sap.ui.viewExtensions": {

    "cross.fnd.fiori.inbox.view.S2": {

        "CustomerExtensionForObjectListItem": {

            className: "sap.ui.core.Fragment",

            fragmentName:

"cross.fnd.fiori.inbox_ext.frag.S2_CustomerExtensionForObjectListItem",

            type: "XML",

        },

    },

    "cross.fnd.fiori.inbox.view.S3": {

        "CustomerExtensionForObjectHeader": {

            className: "sap.ui.core.Fragment",

            fragmentName:

"cross.fnd.fiori.inbox_ext.frag.S3_CustomerExtensionForObjectHeader",

            type: "XML",

        },

    },

},

}
```

i Note

5.2.2 Implement the View Extensions

Create the `frag` folder as described below:

Create Fragment for the List Screen

Use the `CustomerExtensionForObjectListItem` extension point to change the standard list item on the list screen (`S2.view.xml`).

Note

Extending this extension point will overwrite the standard implementation, so we recommend that you copy the standard content in the defined fragment to display the standard properties as well.

Create the `S2_CustomerExtensionForObjectListItem.fragment.xml` file in the `frag` folder (in the `WebContent` folder) as described below:

1. Right-click the `frag` folder, and choose **New** → **File** from the context menu.
2. Enter `S2_CustomerExtensionForObjectListItem.fragment.xml` as the name of the file.
3. Choose **Finish**.
4. Copy the following XML content to this file:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

    <ObjectListItem id="MAIN_LIST_ITEM" type="{device}/listItemType"
press="_handleItemPress"

        title="{TaskTitle}" number="{Value}" numberUnit="{Currency}">

        <attributes>

            <ObjectAttribute id="ATTR1" text="{CreatedByName}" />

            <ObjectAttribute id="ATTR2" text="{path: 'IsEscalated',
formatter: 'cross.fnd.fiori.inbox.Conversions.formatterEscalationState'}" />

        </attributes>

        <firstStatus>

            <ObjectStatus id="STAT1"

                text="{parts: [{path: 'SAP__Origin'}, {path: 'Priority'}]},
formatter: 'cross.fnd.fiori.inbox.Conversions.formatterPriority'}"

                state="{path: 'Priority',
formatter: 'cross.fnd.fiori.inbox.Conversions.formatterPriorityState'}">

            </ObjectStatus>

        </firstStatus>

        <secondStatus>

            <ObjectStatus id="STAT2" text="{path: 'CompletionDeadLine',
type: 'sap.ca.ui.model.type.Date', formatOptions : { style: 'daysAgo'}}" />

        </secondStatus>

    </ObjectListItem>

</core:FragmentDefinition>
```

1 Note

The new fields are defined in the `ObjectListItem` as `number="{Value}" numberUnit="{Currency}"`.

Create Fragment for the Detail Screen

Use the `CustomerExtensionForObjectHeader` extension point to change the standard header part of the detail screen (`S3.view.app.xml`).

Create the `S3_CustomerExtensionForObjectHeader.fragment.xml` file in the `frag` folder (in the `WebContent` folder) as described below:

1. Right-click the `frag` folder, and choose **New → File** from the context menu.
2. Enter `S3_CustomerExtensionForObjectHeader.fragment.xml` as the name of the file.
3. Choose **Finish**.
4. Copy the following XML content to this file:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
    <ObjectHeader title="{TaskTitle}" number="{Value}" numberUnit="{Currency}">
        <attributes>
            <ObjectAttribute id="ARUserName" text="{CreatedByName}" active="true"
press="onEmployeeLaunchTask"></ObjectAttribute>
            <ObjectAttribute text="{path: 'IsEscalated', formatter:
'cross.fnd.fiori.inbox.Conversions.formatterEscalationState'}"></ObjectAttribute>
        </attributes>
        <firstStatus>
            <ObjectStatus
                text="{parts: [{path: 'SAP__Origin'}, {path:
'Priority'}]}, formatter:'cross.fnd.fiori.inbox.Conversions.formatterPriority'}"
                state="{path: 'Priority',
formatter:'cross.fnd.fiori.inbox.Conversions.formatterPriorityState'}">
            </ObjectStatus>
        </firstStatus>
        <secondStatus>
            <ObjectStatus text="{path: 'CompletionDeadLine',
type:'sap.ca.ui.model.type.Date', formatOptions : { style:'daysAgo'}}">
            </ObjectStatus>
        </secondStatus>
    </ObjectHeader>
</core:FragmentDefinition>
```



```
</core:FragmentDefinition>
```

1 Note

The new fields are defined in the ObjectHeader as `number="{Value}" numberUnit="{Currency}"`.

5.3 Create Custom Sorting, Grouping, and Filtering

After you have added the new fields to the S2 screen, you need to create and register a controller extension for the S2 screen. This will contain the custom implementations. Create the `S2.controller.js` file in the `view` folder (in the `WebContent` folder) as described below:

1. Right-click the `view` folder, and choose **New → File** from the context menu.
2. Enter `S2.controller.js` as the name of the file.
3. Choose **Finish**.
4. Add the following code to the `S2.controller.js` file:

```
sap.ui.controller("cross.fnd.fiori.inbox_ext.view.S2", {  
  
});
```
5. Register the extension in the `Component.js` file by adding the highlighted section:

```
metadata: {  
  customizing: {  
    "sap.ui.controllerExtensions": {  
      "cross.fnd.fiori.inbox.view.S2": {  
        "controllerName": "cross.fnd.fiori.inbox_ext.view.S2"  
      }  
    }  
  }  
}
```

5.3.1 Create Custom Grouping

To change the available grouping options, and to enable grouping based on a custom value, implement the `extHookChangeGroupConfig` controller hook in the `S2.controller.js` file as described below. The parameter of the hook is an array, which contains the configuration objects for the available grouping options. To add a new grouping option, add a new object to the array.

```
extHookChangeGroupConfig: function (aGroupConfig) {  
  aGroupConfig.push({
```

```

        key: "Currency",
        textKey: "Currency",
        formatter: function(oContext) {
            switch(oContext.getProperty("Currency")){
                case "EUR": {
                    return "Euro";
                    break;
                }
                case "USD": {
                    return "US Dollar";
                    break;
                }
                default: {
                    return oContext.getProperty("Currency");
                    break;
                }
            }
        }
    });
}

```

5.3.2 Create Custom Filtering

To change the available filtering options, and to add a custom option, implement the `extHookChangeFilterItems` controller hook in the `S2.controller.js` file. A filtering option must have a name (for example, Value), and selectable items (for example, 1000 or more, and less than 1000).

Filtering is done in the back end. Every filter option has a Filter object, which is transformed into the `$filter` parameter of the corresponding OData query URL.

By default, a custom filter object with the `=` operator is created for the custom filter options defined with the `extHookChangeFilterItems` hook (for example, Value = 1000). If it does not suit your business needs, create your own custom filter objects.

This hook method receives an object with all the filter objects and the selected filtering options (`selectedFilterOptions`). Add your custom filter objects to `additionalFilters` as described below:

```

extHookChangeFilterItems: function (aFilters) {

    var valueFilterItem = new sap.m.ViewSettingsFilterItem({

```

```

        text: 'Value',
        multiSelect: false
    });

    var filterValue = this._createFilterItem("Value:High", "1000 or
more");

    valueFilterItem.addItem(filterValue);

    var filterValue = this._createFilterItem("Value:Low", "Less than
1000");

    valueFilterItem.addItem(filterValue);

    aFilters.push(valueFilterItem);
},

extHookGetCustomFilter: function (oFilter) {
    //helper method to remove a filter object
    var fnRemoveFilter = function(filters, path, value){
        var newFilters = filters;
        for(var i=0; i < filters.length; i++){
            o = filters[i];
            if(o.sPath == path && o.oValue1 == value) {
                newFilters.splice(i, 1);
            }
        }
        filters = newFilters;
    };

    if (oFilter.selectedFilterOptions["Value:High"]){
        //remove "Value = High" filter, and add the "Value >= 1000"
filter
        fnRemoveFilter(oFilter.additionalFilters, "Value", "High");

        oFilter.additionalFilters.push(new sap.ui.model.Filter("Value",
sap.ui.model.FilterOperator.GE, 1000));
    }
}

```

```

        if (oFilter.selectedFilterOptions["Value:Low"]){
            //remove "Value = Low" filter, and add the "Value < 1000"
filter
            fnRemoveFilter(oFilter.additionalFilters, "Value", "Low");

            oFilter.additionalFilters.push(new sap.ui.model.Filter("Value",
sap.ui.model.FilterOperator.LT, 1000));
        }

```

5.3.3 Create Custom Sorting

To change the available sorting options, and to add a custom option, implement the `extHookChangeSortItems` controller hook in the `S2.controller.js` file. Make sure that the sorting option ID is the same as the corresponding OData property name.

For example, to enable ascending sorting based on the amount of an item (`Value` property), implement the following code:

```

extHookChangeSortConfig: function (oSortConfig) {
    oSortConfig["Value"] = {
        text: "Amount",
        descending: false,
        getVisible: function() { return true; }
    };
}

```

5.4 Add a List of Line Items to the Detail Screen (S3)

After you have added custom fields to the list and detail screens in the extended app, you can add a list of Shopping Cart line items on the detail screen and enable the processing of shopping carts on item level as well.

Use the following extensibility options:

- `CustomerExtensionForAdditionalDetails` (extension point): to display a list of the line items on the detail screen
- `cross.fnd.fiori.inbox.view.S3` (controller extension) to enable item-level approval

5.4.1 Define the S3 View Extensions in Component.js

In the `WebContent` folder of the project, open the `Component.js` file in the extended app, and add the highlighted objects under `metadata.customizing`:

```
"sap.ui.viewExtensions": {
  "cross.fnd.fiori.inbox.view.S3": {
    "CustomerExtensionForAdditionalDetails": {
      className: "sap.ui.core.Fragment",
      fragmentName: "cross.fnd.fiori.inbox_ext.frag.S3",
      type: "XML"
    },
    "CustomerExtensionForObjectHeader": {
      className: "sap.ui.core.Fragment",
      fragmentName:
cross.fnd.fiori.inbox_ext.frag.S3_CustomerExtensionForObjectHeader",
      type: "XML"
    },
  },
}
[...]
```

```
"sap.ui.controllerExtensions": {
  "cross.fnd.fiori.inbox.view.S3": {
    "controllerName": "cross.fnd.fiori.inbox_ext.view.S3"
  }
},
```

5.4.2 Extend the S3 View with the List of Line Items

In the example, the fragment XML files are stored in the `frag` folder, created under the `WebContent` folder in the extended app.

Create the `S3.fragment.xml` file in the `frag` folder (in the `WebContent` folder) as described below:

1. Right-click the `frag` folder, and choose **New** → **File** from the context menu.
2. Enter `S3.fragment.xml` as the name of the file.
3. Choose **Finish**.
4. Copy the following XML content to this file:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
  <Table items="{detail}/SCAItems/results">
    <headerToolbar>
      <Toolbar>
        <Label text="Items"/>
      </Toolbar>
    </headerToolbar>
    <columns>
      <Column>
      </Column>
      <Column>
        <Label text="Description"/>
      </Column>
    </columns>
  </Table>
</core:FragmentDefinition>
```

```

        </Column>
        <Column>
            <Label text="Quantity"/>
        </Column>
        <Column>
            <Label text="Subtotal"/>
        </Column>
    </columns>
    <items>
        <ColumnListItem
            type="Navigation"
            press="onShowItemDetails">
            <cells>
                <Switch type="AcceptReject" state="true"
                    change="onItemApproveChanged"
                    visible="{detail>/ApprovalOnItemlevel}"/>
                <ObjectIdentifier title="{detail>Description}"/>
                <ObjectNumber number="{detail>Quantity}"
                    unit="{detail>UnitText}"/>
                <ObjectNumber number="{detail>Price}"
                    unit="{detail>Currency}"/>
            </cells>
        </ColumnListItem>
    </items>
</Table>
</core:FragmentDefinition>

```

The root of the xml is a `FragmentDefinition` that contains a `Table` control. The table has a title (implemented as a `Label` in the `Toolbar`) and four columns. The template of the table rows is defined under the `items` node as a `ColumnListItem` control with four cells.

5.4.3 Extend the S3 Controller to Enable Item-Level Selection and Approval

To enable item-level selection and approval, create the `S3.controller.js` file in the `view` folder (in the `WebContent` folder) as described below:

First, create the `view` folder as described below:

1. Right-click the `WebContent` folder, and choose **New** → **Folder** from the context menu.
2. Enter **view** as the name of the folder.
3. Choose **Finish**.

Then, create the `S3.controller.js` file in the `view` folder (in the `WebContent` folder) as described below:

1. Right-click the `view` folder, and choose **New** → **File** from the context menu.
2. Enter **S3.controller.js** as the name of the file.
3. Choose **Finish**.
4. Add the following code to the `S3.controller.js` file:

```
sap.ui.controller("cross.fnd.fiori.inbox_ext.view.S3", {
```

```
});
```

5. Implement the following methods by adding them to this file:

o extHookGetEntitySetsToExpand hook method

It returns an array of the names of the collections that need to be expanded besides the default collections. The "SCAItems" collection is defined in the service extension to provide the line items. For more information, see [Add the Shopping Cart Line Item List to the Service](#).

```
extHookGetEntitySetsToExpand: function () {  
    return ["SCAItems"];  
}
```

o extHookOnDataLoaded hook method

This method enables the processing of the data requested by the detail screen (S3) right after the service call returns. It stores the properties of the items in a buffer that also stores the selected approved items. The `_oApprovalItems` private property of the controller serves as the buffer.

```
_oApprovalItems: {},  
extHookOnDataLoaded: function (oData) {  
  
    var i, imax, items, item;  
  
  
    this._oApprovalItems.SAP__Origin = oData.SAP__Origin;  
  
    this._oApprovalItems.WorkitemID = oData.InstanceID;  
  
    this._oApprovalItems.ScNumber = oData.ScNumber;  
  
    this._oApprovalItems.Comment = "";  
  
    this._oApprovalItems.SCAApprovalItems = [];  
  
  
    for (i = 0, items = oData.SCAItems.results, imax = items.length; i <  
imax; i++) {  
  
        item = {};  
  
        item.SAP__Origin = oData.SAP__Origin;  
  
        item.WorkitemID = items[i].InstanceID;  
  
        item.ItemNumber = items[i].ItemNumber;  
  
        item.ItemApproved = "X";  
  
        this._oApprovalItems.SCAApprovalItems.push(item);  
  
    }  
  
},
```

o onItemApproveChanged event handler method

It is the event handler of the switch control (defined in section [Implement the View Extension to Display a List of Line Items](#)) which can be used to select the line items to be approved.

```
onItemApproveChanged: function (oEvent) {

    var i, imax;

    var itemNumber =
oEvent.getSource().getBindingContext("detail").getProperty("ItemNumber");

    var itemApproved = oEvent.getParameter("state") ? "X" : "0";

    for (i = 0, imax = this._oApprovalItems.SCAApprovalItems.length; i <
imax; i++) {

        if (this._oApprovalItems.SCAApprovalItems[i].ItemNumber === itemNumber)
        {

            this._oApprovalItems.SCAApprovalItems[i].ItemApproved = itemApproved;

            return;

        }

    }

},
```

o `_getSCAHeaderApprovalCollection` private method

It returns the `_oApprovalItems` object (private property of the controller) that contains the header information of the request, the list of its items, and their approval status in a structure that is required by the approve service.

```
_getSCAHeaderApprovalCollection: function () {

    return this._oApprovalItems;

},
```

o `onShowItemDetails` event handler method of the press event

When you choose a line item, this method navigates you to the line item details (S4) screen.

```
onShowItemDetails: function (oEvent) {
    var viewBindingContext = this.getView().getBindingContext();
    var itemBindingContext =
        oEvent.getSource().getBindingContext("detail");
    var sOrigin = viewBindingContext.getProperty("SAP__Origin");
    var sInstanceID = viewBindingContext.getProperty("InstanceID");
    var itemNumber;

    itemNumber = itemBindingContext.getProperty("ItemNumber");

    this.oRouter.navTo("itemDetail", {
        SAP__Origin: sOrigin,
        InstanceID: sInstanceID,
        ItemNumber: itemNumber
    });
}
```



```
    });  
}
```

5.5 Extend the Application with S4 Screen

To display line item details, you need to extend the application with an additional screen (S4 screen). To do the extension, follow the steps described in the sections below.

5.5.1 Define the Navigation Route to the S4 Screen

In order to provide navigation to the S4 screen, in the `WebContent` folder, extend the `Component.js` file with a `routing` section.

Open the `Component.js` file and add the highlighted lines:

```
cross.fnd.fiori.inbox.Component.extend("cross.fnd.fiori.inbox_ext.Component", {  
  metadata: {  
    routing: {  
      routes: {  
        masterDetail: {  
          subroutes: {  
            master: {  
              subroutes: {  
                "itemDetail": { // 1  
                  pattern: "itemDetail/{SAP__Origin}/{InstanceID}/{ItemNumber}", // 2  
                  viewPath: "cross.fnd.fiori.inbox_ext.view", // 3  
                  view: "S4" // 4  
                }  
              }  
            }  
          }  
        }  
      }  
    }  
  }  
});
```

```

    },
    ... Other metadata declarations ...
  },
  ... Other component declarations ...
});

```

Footnote:

1) The `itemDetail` string is the name of the route. This string is referenced in the S3 controller to select the target during navigation.

2) When navigation happens, the specified pattern is appended to the application URL.

Patterns can contain parameters between curly brackets. In this example, these parameters are the following:

- o `SAP__Origin`
- o `InstanceID`
- o `ItemNumber.`

These parameters are passed from the S3 controller (`navTo` method call, see section [Extend the S3 Controller to Enable Item-Level Selection and Approval](#)) to the S4 controller (see section [Create S4 controller](#)), and they are used to select one line item of a given work item.

3) Module path of the view. It refers to the `view` subdirectory of the project.

4) Name of the view. Full filename of the view is `view/S4.view.xml`.

5.5.2 Create the S4 View

Create the `S4.view.xml` file in the `view` folder (in the `WebContent` folder) as described below:

1. Right-click the `view` folder, and choose **New** → **File** from the context menu.
2. Enter **S4.view.xml** as the name of the file.
3. Choose **Finish**.
4. Copy the following XML content to this file:

```

<sap.ui.core:View controllerName="cross.fnd.fiori.inbox_ext.view.S4" xmlns="sap.m"
xmlns:sap.ui.core="sap.ui.core" xmlns:sap.ui.layout.form="sap.ui.layout.form"
xmlns:sap.ui.layout="sap.ui.layout">

  <Page id="page" title="{/Description}">

    <content>

      <ObjectHeader title="{/Description}" number="{/Value}" numberUnit="{/Currency}">

```

```

<attributes>

  <ObjectAttribute text="{parts: [{path: '/Quantity'}, {path: '/UnitText'}]},
formatter: 'cross.fnd.fiori.inbox_ext.util.Formatter.formatQuantity'"/>

  <ObjectAttribute text="{parts: [{path: '/Price'}, {path: '/Currency'}, {path:
'/PriceUnit'}, {path: '/UnitText'}]}, formatter:
'cross.fnd.fiori.inbox_ext.util.Formatter.formatPrice'"/>

</attributes>

</ObjectHeader>

<sap.ui.layout.form:SimpleForm minWidth="1024" editable="false">

  <sap.ui.layout.form:content>

    <sap.ui.core:Title text="Information"/>

    <Label text="Product">

      <layoutData>

        <sap.ui.layout:ResponsiveFlowLayoutData minWidth="192" weight="3"/>

      </layoutData>

    </Label>

    <Text text="{/Description}">

      <layoutData>

        <sap.ui.layout:ResponsiveFlowLayoutData weight="5"/>

      </layoutData>

    </Text>

    <Label text="Product Category">

      <layoutData>

        <sap.ui.layout:ResponsiveFlowLayoutData minWidth="192" weight="3"/>

      </layoutData>

    </Label>

    <Text text="{/CategoryDescription}">

      <layoutData>

        <sap.ui.layout:ResponsiveFlowLayoutData weight="5"/>

      </layoutData>

    </Text>

```

```
</sap.ui.layout.form:content>

</sap.ui.layout.form:SimpleForm>

</content>

</Page>

</sap.ui.core:View>
```

Footnote:

- 1) The `controllerName` attribute of the `View` element specifies which controller is associated to the S4 view. The full filename of the controller is `view/S4.controller.js`.
- 2) The view uses data binding to display values. The model is created by the S4 controller and then it is assigned to the view. You can reference individual properties of the model using curly brackets.
- 3) Two `ObjectAttribute` elements use custom formatters (`formatQuantity` and `formatPrice`) to format properties before displaying them. You can define these formatters in a separate file (see section [Create formatters](#)).
- 4) If you need multi-language support, move the following labels to an external i18n resource:
 - o Information
 - o Product
 - o Product Category

5.5.3 Create the S4 Controller

Create the `S4.controller.js` file in the `view` folder (in the `WebContent` folder) as described below:

1. Right-click the `view` folder, and choose **New** → **File** from the context menu.
2. Enter `S4.controller.js` as the name of the file.
3. Choose **Finish**.
4. Copy the following content to this file:

```
jQuery.sap.require("sap.ca.scfld.md.controller.BaseDetailController");
jQuery.sap.require("sap.ca.ui.utils.busydialog");

sap.ca.scfld.md.controller.BaseDetailController.extend("cross.fnd.fiori.inbox_ext.v
iew.S4", {
```

```

onInit: function() {

    // Execute onInit of base class.

    sap.ca.scfld.md.controller.BaseDetailController.prototype.onInit.call(this);

    // Setting the navigate back visible true for moving back to S3 controller.

    this.getView().byId("page").setShowNavButton(true);

    // Get DataManager instance.

    this.oDataManager =
sap.ca.scfld.md.app.Application.getImpl().getComponent().getDataManager();    // 1

    this.oRouter.attachRouteMatched(function(oEvent) {

        if (oEvent.getParameter("name") == "itemDetail") {    // 2

            // This code will be executed when the user navigates to S4.

            // Read parameters.

            var args = oEvent.getParameter("arguments");    // 3

            var sOrigin = args.SAP__Origin;

            var sInstanceID = args.InstanceID;

            var sItemNumber = args.ItemNumber;

            // Execute line item query.

            this.readItem(sOrigin, sInstanceID, sItemNumber,    // 4

                jQuery.proxy(this.readItemSuccess, this),

                null);

        }

    }, this);

},

readItem: function(sOrigin, sInstanceID, sItemNumber, onSuccess, onError) {    // 5

    var that = this;

    sap.ca.ui.utils.busydialog.requireBusyDialog();

```

```

// Assemble GET request and execute it.

this.oDataManager.oDataRead("/SCAItemCollection(SAP__Origin='" +
    jQuery.sap.encodeURL(sOrigin) +
    "',InstanceID='" + jQuery.sap.encodeURL(sInstanceID)
+
    "',ItemNumber='" + jQuery.sap.encodeURL(sItemNumber)
+
    "')",
    null,
    null,
    true,
    function(oData, oResponse) {
        sap.ca.ui.utils.busydialog.releaseBusyDialog();
        if (oData) {
            if (onSuccess)
                onSuccess(oData);
        }
    },
    function(oError) {
        sap.ca.ui.utils.busydialog.releaseBusyDialog();
        that.oDataManager.oDataRequestFailed(oError);
        if (onError)
            onError(oError);
    });
},

readItemSuccess: function(oData) {
    // Set model of the view.

    var oModel = new sap.ui.model.json.JSONModel(oData);
    this.getView().setModel(oModel);
}
});

```

Footnote:

- 1) Get reference to `DataManager` instance. This object is used to perform OData requests to Gateway.
- 2) When navigation happens in the application, a `routeMatched` event is fired. By reading the `name` parameter of the event, you can query the name of the triggered route (`itemDetail`), which has to match the route definition (see section [Define the Navigation Route to the S4 Screen](#)).
- 3) Parameters passed to the S4 screen can be read from the object returned by `oEvent.getParameter("arguments")`. The received parameters are the following:
 - o `SAP__Origin`
 - o `InstanceID`
 - o `ItemNumber`
- 4) During view initialization, a call to `readItem` is performed, which executes a line-item request to Gateway. Upon successful execution, `readItemSuccess` is called.
- 5) Implementation of `readItem`. This method assembles a line-item request from the parameters listed above, executes the request, and calls either the `onSuccess` or the `onError` callback, depending on the Gateway response. The most important method calls are the following:
 - o `DataManager.oDataRead`: responsible for request execution
 - o `requireBusyDialog` and `releaseBusyDialog`: show and hide progress indicator
 - o `DataManager.oDataRequestFailed`: displays error dialog in case of Gateway error
- 6) Creates a JSON model from the Gateway response and assigns it to the view. Data-binding can be used to reference individual properties of the model in the view.

5.5.4 Create Formatters

Custom formatters reside in `util/Formatter.js`.

First, create the `util` folder as described below:

1. Right-click the `WebContent` folder, and choose **New** → **Folder** from the context menu.
2. Enter `util` as the name of the folder.
3. Choose **Finish**.

Then, create the `Formatter.js` file in the `util` folder (in the `WebContent` folder) as described below:

1. Right-click the `util` folder, and choose **New** → **File** from the context menu.
2. Enter `Formatter.js` as the name of the file.
3. Choose **Finish**.
4. Copy the following content to this file:

```

jQuery.sap.declare("cross.fnd.fiori.inbox_ext.util.Formatter");

cross.fnd.fiori.inbox_ext.util.Formatter = {

    formatQuantity: function (quantity, unitText) {

        if (quantity != null && unitText != null) {

            return quantity + " " + unitText;

        }

    },

    formatPrice: function (price, currency, priceUnit, unitText) {

        if (price != null && currency != null && priceUnit != null && unitText
!= null) {

            return price + " " + currency + "/" + priceUnit + " " +
unitText;

        }

    }

};

```

To ensure that formatters are loaded before the S4 view calls them, add a `require` declaration to the `Component.js`:

```

jQuery.sap.require("cross.fnd.fiori.inbox_ext.util.Formatter");

cross.fnd.fiori.inbox.Component.extend("cross.fnd.fiori.inbox_ext.Component", {

    ... Other declarations ...

});

```

5.6 Add a New Tab to the Detail Screen

To display the list of involved approvers (chain of approvers), you must extend the detail screen with a new tab. To do so, implement the `CustomerExtensionForAdditionalDetails` extension point of the S3 screen.

5.6.1 Extend the S3 View with a New Tab

Create the `S3_CustomerExtensionForAdditionalTabs.fragment.xml` file in the `frag` folder (in the `WebContent` folder) as described below:

1. Right-click the [view](#) folder, and choose **New → File** from the context menu.
2. Enter **S3_CustomerExtensionForAdditionalTabs.fragment.xml** as the name of the file.
3. Choose **Finish**.
4. Add the following extension to this file:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

    <IconTabFilter id="SCAHeaderApprovalChain"

        icon="sap-icon://group"

        visible="{path: 'detail>/SCAApproversHeader/results/length',
formatter: 'cross.fnd.fiori.inbox.Conversions.formatterIsNotZero' }">

        <List inset="true" showSeparators="None"
items="{detail>/SCAApproversHeader/results}">

            <items>

                <StandardListItem title="{detail>ApproverFullName}"
type="Active" iconInset="false" press=""></StandardListItem>

            </items>

        </List>

    </IconTabFilter>

</core:FragmentDefinition>
```

Footnote:

The OData service provides the list of approvers in the `SCAApproversCollection` property and the name of approvers in the `ApproverFullName` property.

5.6.2 Declare the New Tab

Open the `Component.js` file and add the highlighted objects under `metadata.customizing` (if they do not exist yet):

```
customizing: {

    "sap.ui.viewExtensions": {

        "cross.fnd.fiori.inbox.view.S3": {

            "CustomerExtensionForAdditionalTabs": {

                className: "sap.ui.core.Fragment",
```

```

                                fragmentName:
"cross.fnd.fiori.inbox_ext.frag.S3_CustomerExtensionForAdditionalTabs",
                                type: "XML"
                            },
                        }
                    },
                },
            },
        },
    },
}

```

5.7 Enable Item-Level Approval

The `ApprovalOnItemLevel` property in the Task collection, in the Gateway, defines the decision level for a given work item list. If the value of the property is `TRUE`, you can perform item-level approval; if it is `FALSE`, you can perform header-level approval.

In case of header-level approval, the [Approve](#) and [Reject](#) buttons are displayed (based on your customizing). In case of item-level approval, when you make a decision for every item in the list, a [Send](#) button is displayed in the footer bar, instead of the [Approve](#) and [Reject](#) buttons.

To enable item-level approval in the front end, follow the steps below:

5.7.1 Change the Action Buttons

In the standard application a hook method is available to change the buttons in the footer bar. The name of the hook method is `extHookChangeFooterButtons`. The method has only one parameter, which is an object with three properties:

- `PositiveAction` - object that contains the positive action button (for example, [Approve](#))
- `NegativeAction` - object that contains the negative action button (for example, [Reject](#))
- `ButtonList` - array that contains all other buttons (for example, [Forward](#) and [Claim](#))

In the following code snippet, first a check is performed whether item-level approval is available. If item level approval is available, the positive and negative action buttons are removed, and the [Send](#) button is added to the array. To handle the Send event, the `showItemApprovalDialog` function is attached.

To change the action buttons, enter the following code in the S3 Controller (`WebContent/view/S3.controller.js`):

```

        extHookChangeFooterButtons: function(oButtonList) {

            var bApproveOnItemLevel =
this.getView().getBindingContext().getProperty("ApprovalOnItemlevel"); // 1

            if (bApproveOnItemLevel==true) { //2

                oButtonList.oPositiveAction = null;
            }
        }
    }
}

```

```

oButtonList.oNegativeAction = null;

var that = this;

oButtonList.aButtonList.unshift({
    sBtnTxt : "Send",
    onBtnPressed : function(event) {
        that.showItemApprovalDialog(); // 3
    }
});
},

```

Footnote:

1. Checks whether item-level approval is available
2. If item level approval is available, the positive and negative action buttons are removed, and the [Send](#) button is added to the array
3. To handle the Send event, the `showItemApprovalDialog` function is attached.

5.7.2 Implement Event Handler for the Send Button

Previously, you have attached the `showItemApprovalDialog` function to handle the Send event.

When the user chooses the [Send](#) button, a confirmation dialog box is displayed with a text area for comments, along with the [Submit](#) and [Cancel](#) buttons.

Note

To create this dialog box, we suggest that you use the standard SAP confirmation dialog reusable component.

If the user chooses the [Submit](#) button, the `performCreate` function is called, which posts the items to the back end along with the decision the user has made. The `performCreate` function has the following parameters that you need to set:

- `sPath`

It is the exact path in the model we would like to create, that is, the name of the collection. The POST request uses this collection in the request body. For more information, see section [Extending the Service Implementation](#).

- `SAP_Origin`

`SAP_Origin` is the system alias which defines the back-end system. It is passed to the `performCreate` function.

- `sInstanceID`

ID of the work item. It is passed to the `performCreate` function.

- `oData`

It contains the header and item information of the work item. It is passed to the `performCreate` function. The easiest way to put the `oData` together is to call the `_getSCAHeaderApprovalCollection` private function which returns the necessary data in the appropriate format.

- `onSuccess`

Success callback function

- `onError`

Error callback function

Call the `performCreate` function as described below:

```
this.performCreate("SCAHeaderApprovalCollection", sOrigin, sInstanceID,  
this._getSCAHeaderApprovalCollection(), ...)
```

To implement the event handler, enter the following code in the S3 Controller:

```
showItemApprovalDialog : function() {  
    sap.ca.ui.dialog.confirmation.open({  
        question : "Send",  
        showNote : true,  
        title : this.i18nBundle.getText("XTIT_SUBMIT_DECISION"),  
        confirmButtonLabel :  
this.i18nBundle.getText("XBUT_SUBMIT"),  
        noteMandatory: false,  
    }, $.proxy(function(oResult) {  
        if(oResult.isConfirmed===true){  
            var viewBindingContext =  
this.getView().getBindingContext();  
            var sOrigin =  
viewBindingContext.getProperty("SAP__Origin");  
            var sInstanceID =  
viewBindingContext.getProperty("InstanceID");  
  
            this.performCreate("SCAHeaderApprovalCollection",  
sOrigin, sInstanceID, this._getSCAHeaderApprovalCollection(),  
$.proxy(function(oData) {  
  
                jQuery.sap.delayedCall(500, this,  
function() {
```

```

        sap.ca.ui.message.showMessageToast(this.i18nBundle.getText("dialog.success.complete"));

        });

        }, this), function(oError) {

            //implement error handling here if

required

        }

    );

}

},

this)

);

},

```

5.7.3 Implement the performCreate Function

For the item-level approval the [Send](#) button was introduced. To enable item-level approval, implement the `performCreate` function.

Unlike the other actions, the [Send](#) action has to have the origin in the URL. Create an own `oDataModel`, add the origin to the URL, and use the create function in the `oDataModel` to send the request.

To implement the `performCreate` function, enter the following code in the S3 Controller (`WebContent/view/S3.controller.js`):

```

performCreate: function (sPath, sOrigin, sInstanceID, oData, onSuccess, onError)
{

    sap.ca.ui.utils.busydialog.requireBusyDialog();

    // we need to use the origin in the url, so own model is needed for the create

    var sPostServiceUrl = this.getView().getModel("POSTACTION").sServiceUrl;

    if (sPostServiceUrl.indexOf(";mo") !== -1) {

        var urlParts = sPostServiceUrl.split(";mo");

        sPostServiceUrl = urlParts[0] + ";o=" + sOrigin + urlParts[1];

    }

    var oPostActionModel = new sap.ui.model.odata.ODataModel(sPostServiceUrl,
true);

    oPostActionModel.setUseBatch(true);

```

```

var oSettings = {
    success: $.proxy(function(oData, oResponse) {
        sap.ca.ui.utils.busydialog.releaseBusyDialog();
        $.sap.log.info("successful action");
        this.oDataManager.processListAfterAction(sOrigin,
sInstanceID);

        if (onSuccess) {
            $.proxy(onSuccess(oData, oResponse), this);
        }
    }, this),
    error: $.proxy(function(oError) {
        sap.ca.ui.utils.busydialog.releaseBusyDialog();
        this.oDataManager.processListAfterAction(sOrigin,
sInstanceID);

        if (onError)
            onError(oError);
    }, this),
    async: true
};

oPostActionModel.create(sPath, oData, oSettings);
},

```

5.8 Limit the Forward Agent List Size

You can define the maximum number of agents listed in the [Forward](#) dialog by implementing the `extHookChangeListSizeLimit` controller hook.

Create the `Forward.controller.js` file in the view folder (in the `WebContent` folder) as described below:

1. Right-click the view folder, and choose **New** → **File** from the context menu.
2. Enter **Forward.controller.js** as the name of the file.
3. Choose **Finish**.
4. Add the following code to the `S3.controller.js` file:

```

sap.ui.controller("cross.fnd.fiori.inbox_ext.view.Forward", {

```

```
});
```

5. Implement the following method by adding it to this file:

```
extHookChangeListSizeLimit: function() {  
    // limit the number of agents to 5  
    return 5;  
}
```

6. Register the extension in the `Component.js` file by adding the highlighted section:

```
metadata: {  
    customizing: {  
        "sap.ui.controllerExtensions": {  
            "cross.fnd.fiori.inbox.view.Forward": {  
                "controllerName":  
                "cross.fnd.fiori.inbox_ext.view.Forward"  
            }  
        }  
    }  
}
```

6 Provide Extensions for a Specific Task Type in Detail View

In order to create different extensions for a particular task type, you need to map a task type with a new view route. The new view will extend the code from the present detail view and then new extensions for the extended view will be defined.

For the current scenario, we will implement the `CustomerExtensionForAdditionalDetails` extension as used in the Shopping Cart scenario defined in section [Add a List of Line Items to the Detail Screen \(S3\)](#) but without providing any item-level approval for a particular task type.

Make sure to fulfill the following prerequisites:

1. The back-end system used in the Shopping Cart scenario is configured and the tasks are retrieved as defined in the Shopping Cart scenario.
2. Multiple back-end systems exist, providing different types of tasks.
3. The back-end configuration of all systems is the same as that of the Shopping Cart scenario. The service extension is defined as given in section [Extending in the Gateway](#).
4. The application is extended, however no extension points are defined.

6.1 Create Mapping of Task Types with the New View

To create new extensions for a task type, you need to create the mapping of `TaskDefinitionID` of task type with the new view route.

1. In the [WebContent](#) folder of the extended application, create a new file with the name **`TaskExtensionMappingConfig.json`**. For more information on how to create extended application, see section [Create an Extended App Skeleton](#).
2. Copy the following code with a valid `TaskDefinitionId` in the above created file.

```
{  
  
    "TS92400373": "additionalDetails"  
  
}
```

This code defines the mapping between a task type by its `TaskDefinitionID` and a route created for a new view.

6.2 Extend the S2 Controller

To enable task-based extensions, you need to extend the S2 controller. With this you can assign different views based on the task type to enable displaying of views based on a task type.

6.2.1 Create the S2 Controller

1. In the **Web Content** folder of the extended application, under the **View** folder, create a new file, **S2.controller.js**.
2. Copy the following code in the file.

```
sap.ui.controller("cross.fnd.fiori.inbox_ext.view.S2", {

    onInit: function() {

        var that = this;

        // Read the configuration file for the detail page routes

        var sUrl = jQuery.sap.getModulePath("cross.fnd.fiori.inbox_ext") +
            "/TaskExtensionMappingConfig.json";

        that.oDetailRouteConfig = jQuery.sap.sjax({url: sUrl, dataType:
            "json"}).data || {};

    },

    getDetailRouteName : function() {

        if (!this.oDetailRouteConfig) {

            return "detail";

        }

        var oItem = this.getList().getSelectedItem();

        if (!oItem) {

            return null;

        }

        var oItemData =
            oItem.getBindingContext().getModel().getData(oItem.getBindingContext().getPath()
            );
    }
});
```

```

        var sDetailRouteName =
this.oDetailRouteConfig[oItemData.TaskDefinitionID];

```

```

        if (sDetailRouteName) {

            return sDetailRouteName;

        } else {

            return "detail";

        }

```

```

    }

```

```

});

```

The above code fetches data from the `TaskExtensionMappingConfig.json` file and fetches the view route required if the task requires extension.

6.2.2 Provide Controller Extension for S2 in the Component.js

The `Component.js` file in the extended application does not contain any extension in the customizing file. Copy the following code in the [Customizing](#) section of the `Component.js` file to provide controller extension for the S2 controller created in the previous step for the extended application.

```

customizing: {

"sap.ui.controllerExtensions": {

"cross.fnd.fiori.inbox.view.S2": {

"controllerName": "cross.fnd.fiori.inbox_ext.view.S2"

},

},

},

```

6.3 Add a New View for a Specific Task Type

To enable task type based extensions, you need to create separate views for each task type along with their controllers.

6.3.1 Create a New View for the Task Type

1. In the **View** folder of the extended application, create a new file, **S3_additionalDetails.view.xml**.
2. Copy the code from the **S3.view.xml** file from the base application and paste it in the new file.
3. In the **S3_additionalDetails.view.xml** file, replace
controllerName=**cross.fnd.fiori.inbox.view.S3** with
controllerName=**cross.fnd.fiori.inbox_ext.view.S3_additionalDetails**.

6.3.2 Create a New Controller for the Task Type

1. In the **View** folder of the extended application, create a new file, **S3_additionalDetails.controller.js**.
2. Copy the following code in the file:

```
jQuery.sap.require({modName:"cross.fnd.fiori.inbox.view.S3", type:
"controller"});

cross.fnd.fiori.inbox.view.S3.extend("cross.fnd.fiori.inbox_ext.view.S3_addition
alDetails", {

    handleNavToDetail: function(oEvent){

        if (oEvent.getParameter("name") === "additionalDetails") {

            var sInstanceID = oEvent.getParameter("arguments").InstanceID;

            if (sInstanceID.search(":") == (sInstanceID.length - 1)) {

                return;

            }

            // Deep link scenario: if the detail navigation happens before
            the S2 list was downloaded, navigate to a different URL

            // so when the list data arrives and the item gets selected the
            URL will change, and the navigation won't be stopped

            if (jQuery.isEmptyObject(this.getView().getModel().oData)) {

                var oParameters = {
```

```

        SAP__Origin :
oEvent.getParameters().arguments.SAP__Origin,

        InstanceID :
oEvent.getParameters().arguments.InstanceID + ":",

        contextPath :
oEvent.getParameters().arguments.contextPath

    };

    this.oRouter.navTo("additionalDetails", oParameters,
true);

    return;

}

//In case of a list item selection the first tab shall be
selected

//Exception: Comment is added on the comment tab - this tab
must stay selected or nav to detail on phone

if (!this.stayOnDetailScreen || jQuery.device.is.phone) {
    var oDescriptionTab = this.oTabBar.getItems()[0];
    this.oTabBar.setSelectedItem(oDescriptionTab);
} else {
    this.stayOnDetailScreen = false;
}

var oRefreshData = {
    sCtxPath: "/" +
oEvent.getParameters().arguments.contextPath,

    sInstanceID: sInstanceID,

    sSAP__Origin:
oEvent.getParameter("arguments").SAP__Origin,

    bCommentCreated: false

};

this.refreshData(oRefreshData);

}

},

```

```
});
```

6.3.3 Map the New View in Component.js

To create a route for `additionalDetails`, copy the code below before the [Customizing](#) section in `Component.js` file:

```
routing: {  
  
    routes: {  
  
        masterDetail: {  
  
            subroutes: {  
  
                master: {  
  
                    subroutes: {  
  
                        "additionalDetails": {  
  
                            "pattern": "additionalDetails  
/{SAP__Origin}/{InstanceID}/{contextPath}",  
  
                            "viewPath":  
"cross.fnd.fiori.inbox_ext.view",  
  
                            "view": "S3_additionalDetails"  
  
                        }  
  
                    }  
  
                }  
  
            }  
  
        }  
  
    },  
},
```

6.4 Add Extension for the Specific Task Type

You can write a fragment now for the required task type by creating a new fragment and mapping it to the newly created view.

6.4.1 Write the New Extension Fragment

1. First, in the [Web Content](#) folder, create a folder, called `frag`.
2. To create a new extension for the new view, create the file `S3_additionalDetails.fragment.xml` in the `frag` folder.

Copy the following code in the file.

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
  <Table items="{detail>SCAItems/results}">
    <headerToolbar>
      <Toolbar>
        <Label text="Items"/>
      </Toolbar>
    </headerToolbar>
    <columns>

      <Column>
        <Label text="Description"/>
      </Column>
      <Column>
        <Label text="Quantity"/>
      </Column>
      <Column>
        <Label text="Subtotal"/>
      </Column>
    </columns>
    <items>
      <ColumnListItem
        type="Navigation"
        >
        <cells>

          <ObjectIdentifier title="{detail>Description}"/>
          <ObjectNumber number="{detail>Quantity}"
            unit="{detail>UnitText}"/>
          <ObjectNumber number="{detail>Price}"
            unit="{detail>Currency}"/>
        </cells>
      </ColumnListItem>
    </items>
  </Table>
</core:FragmentDefinition>
```

6.4.2 Provide Extensions for the New Fragment in the Component.js

In **Component.js**, under the **Customizing** section add the following extension:

```
"sap.ui.viewExtensions": {  
    "cross.fnd.fiori.inbox_ext.view.S3_additionalDetails": {  
        "CustomerExtensionForAdditionalDetails": {  
            className: "sap.ui.core.Fragment",  
            fragmentName:  
"cross.fnd.fiori.inbox_ext.frag.S3_additionalDetails",  
            type: "XML"  
        },  
    },  
},
```

6.5 Extend the S3_additionDetails Controller

In order to provide the data required for the extension fragment, you need to implement the `extHookGetEntitySetsToExpand` hook method.

6.5.1 Write the S3_additionDetails Controller Extension

1. In the **Web Content** folder, under the **View** folder, create a file with the name `S3_additionDetailsExtn.controller.js`.
2. Copy the following code in that file.

```
sap.ui.controller("cross.fnd.fiori.inbox_ext.view.S3_additionDetailsExtn", {  
  
    extHookGetEntitySetsToExpand: function () {  
        return ["SCAItems"];  
    },
```

```
});
```

This hook method returns an array of the names of the collections that need to be expanded besides the default collections. The `SCAItems` collection is defined in the service extension in section [Extending the Service Implementation](#) to provide the line items. This method can be used to expand any custom collection that the user wants to use.

6.5.2 Define S3_additionalDetails Controller Extension in the Component.js

After writing the `S3_additionalDetails` controller extension in the [Controller extension](#) section of `Component.js`, the code looks as follows:

```
cross.fnd.fiori.inbox.Component.extend("cross.fnd.fiori.inbox_ext.Component",{

    metadata: {

        version : "1.0",

        customizing: {

            "sap.ui.viewExtensions": {

                "cross.fnd.fiori.inbox_ext.view.S3_additionalDetails": {

                    "CustomerExtensionForAdditionalDetails": {

                        className: "sap.ui.core.Fragment",

                        fragmentName:

"cross.fnd.fiori.inbox_ext.frag.S3_additionalDetails",

                        type: "XML"

                    },

                },

            },

        },

    },

});
```



```

"sap.ui.controllerExtensions": {
    "cross.fnd.fiori.inbox_ext.view.S3_additionalDetails": {
        "controllerName":
"cross.fnd.fiori.inbox_ext.view.S3_additionDetailsExtn"
    },
    "cross.fnd.fiori.inbox.view.S2": {
        "controllerName":
"cross.fnd.fiori.inbox_ext.view.S2"
    },
},

routing: {
    routes: {
        masterDetail: {
            subroutes: {
                master: {
                    subroutes: {

                        "additionalDetails": {
                            "pattern": "additionalDetails
/{SAP__Origin}/{InstanceID}/{contextPath}",
                            "viewPath":
"cross.fnd.fiori.inbox_ext.view",
                            "view": "S3_additionalDetails"
                        }
                    }
                }
            }
        }
    },
},

```

```
}  
});
```

7 Configuring the Launchpad

After extending the standard app in the Gateway and on the front end as well, you need to add a tile for the extended app in the Launchpad.

Assign the end user to the `SAP_FND_BCR_MANAGER_T` role to use the Manager Approval (Cross) – Content (SAP_FND_BC_MANAGER_T) catalog. Create the new objects in the `SAP_FND_TC_T` catalog, and then copy them as reference to the `SAP_FND_BC_MANAGER_T` catalog.

Note

`SAP_FND_TC_T` is a technical catalog, and serves as a repository for all cross applications. The `SAP_FND_BC_MANAGER_T` catalog contains only references from the technical catalog, and this is what the end user can access.

7.1 Create Target Mapping

1. Open the Launchpad administration web interface.
2. Select the My Inbox (`SAP_FND_TC_T`) catalog.
3. Select [Target Mapping](#).
4. Choose the [Create Target Mapping](#) button.
5. Enter the following data:
 - Semantic Object: **WorkflowTask**
 - Action: <action name that you use to create the tile, for example, **My Inbox Cart Approval Extension**>

In this example, use **displayInboxExt**.

- Select **UI5 SAP Fiori Application** as **Source of the Navigation Target**
- Title: <enter the title that will appear on the screen>
- URL: enter **/sap/bc/ui5_ui5/sap/<BSP application name>**.

Use the same BSP application name that you have selected when you have uploaded the project to the Gateway system in section [Upload the New Project](#).

- Component: **=<SAPUI5 component name>**

For the SAPUI5 component name, see section [Create an Extended App Skeleton](#).

In this example, use the **cross.fnd.fiori.inbox_ext** SAP UI5 component.

6. Choose [Save](#).
7. Copy the Target Mapping as reference to the `SAP_FND_BC_MANAGER_T` catalog.

7.2 Create an App Launcher Tile

1. Open the Launchpad administration web interface.
2. Select the My Inbox (SAP_FND_TC_T) catalog.
3. Choose the [Add Tile](#) button (the last tile with a plus sign)
4. Select [App Launcher - Static](#).
5. Choose the newly created tile.
6. Enter a title, subtitle, and select an icon for the tile.
7. In the [Navigation](#) section, select the [Use semantic object navigation](#) checkbox and enter the following:
 - Semantic Object: **WorkflowTask**
 - Action: <the action name that you have already defined for target mapping in section [Create Target Mapping](#)>. In this example, use **displayInboxExt**.
 - Parameters: scenarioId=<Scenario Identifier>
8. Choose [Save](#).
9. Copy the tile as reference to the SAP_FND_BC_MANAGER_T catalog.

Appendix

Appendix A - Source Code of the RFC Function Modules for the Shopping Cart Approval Scenario

To be able to exchange data between your back-end system and Gateway, you need to create RFC function modules and the related DDIC structures in your back-end system.

Use the custom RFCs (and related DDIC structures) listed below to display Shopping Cart details. These RFCs require the software component GBSRM001.

ZCARTAPP_GET_SC_DETAILS - Get Shopping Cart Details

```
FUNCTION ZCARTAPP_GET_SC_DETAILS .
  *-----
  *""Local Interface:
  *  IMPORTING
  *    VALUE(IV_WORKITEM_ID) TYPE  SWW_WIID
  *  EXPORTING
  *    VALUE(ES_HEADER) TYPE  /GBSRM/S_APV_SC_HD
  *    VALUE(ET_ITEM) TYPE  /GBSRM/T_APV_SC_IT
  *    VALUE(ET_ACCOUNTING) TYPE  /GBSRM/T_APV_SC_AC
  *    VALUE(ET_APPROVER) TYPE  /GBSRM/T_APV_SC_APPROVER
  *-----

  DATA:
    lo_api TYPE REF TO /gbsrm/cl_apv_sc_api,
    lo_msg_container TYPE REF TO /iwbsp/if_message_container,
    lo_msg TYPE REF TO /IWBEF/CL_MGW_MSG_CONTAINER.

  lo_msg_container ?= /IWBEF/CL_MGW_MSG_CONTAINER=>GET_MGW_MSG_CONTAINER( ).
  lo_api = /gbsrm/cl_apv_sc_api=>get_instance( ).

  " Get Data from API class
  CALL METHOD lo_api->get_sc_details
    EXPORTING
      iv_workitem_id = iv_workitem_id
      iv_user_name = sy-uname
      io_msg_container = lo_msg_container
    IMPORTING
      es_header = es_header
      et_item = et_item
      et_accounting = et_accounting
      et_approver = et_approver.

ENDFUNCTION.
```

ZCARTAPP_GET_WFTASKLIST - Get Shopping Cart Workflow Task List

```
FUNCTION ZCARTAPP_GET_WFTASKLIST .
*"-----
***"Local Interface:
*"  IMPORTING
*"      VALUE(IT_FILTER_SELECT_OPTIONS) TYPE  /IWBEP/T_MGW_SELECT_OPTION
*"      OPTIONAL
*"      VALUE(IS_PAGING) TYPE  /IWBEP/S_MGW_PAGING OPTIONAL
*"  EXPORTING
*"      VALUE(ET_LIST_DATA) TYPE  /GBSRM/T_APV_SC_WF_LIST
*"-----

DATA:
      lo_api                      TYPE REF TO /gbsrm/cl_apv_sc_api.
lo_api = /gbsrm/cl_apv_sc_api=>get_instance( ).

CALL METHOD lo_api->get_list
EXPORTING
      iv_user_name                = sy-uname
      it_filter_select_options    = it_filter_select_options
      is_paging                  = is_paging
IMPORTING
      et_sc_list_data            = et_list_data.

ENDFUNCTION.
```

ZCARTAPP_ITEMLEVEL_APPROVAL - Shopping Cart Item-Level Approval

```
FUNCTION ZCARTAPP_ITEMLEVEL_APPROVAL.
*"-----
***"Local Interface:
*"  IMPORTING
*"      VALUE(IV_WORK_ITEM) TYPE  SWW_WIID
*"      VALUE(IV_APPROVAL_NOTE) TYPE  STRING
*"      VALUE(IT_ITEM_DECISION) TYPE  /GBSRM/T_APV_SC_IBA_IT
*"-----

DATA: lo_api                      TYPE REF TO /gbsrm/cl_apv_sc_api,
      lo_msg_container           TYPE REF TO /iwbsp/if_message_container.

DATA: ls_iba_hdr_exp            TYPE /gbsrm/s_apv_sc_iba_hd_x_it,
      lt_item_decision          TYPE /gbsrm/t_apv_sc_iba_it.

DATA: lv_workitem_id            TYPE sww_wiid,
      lv_approval_note          TYPE string,
      lv_user_name              TYPE syuname.

lo_api = /gbsrm/cl_apv_sc_api=>get_instance( ).
lo_msg_container = /iwbsp/cl_mgw_msg_container=>get_mgw_msg_container( ).

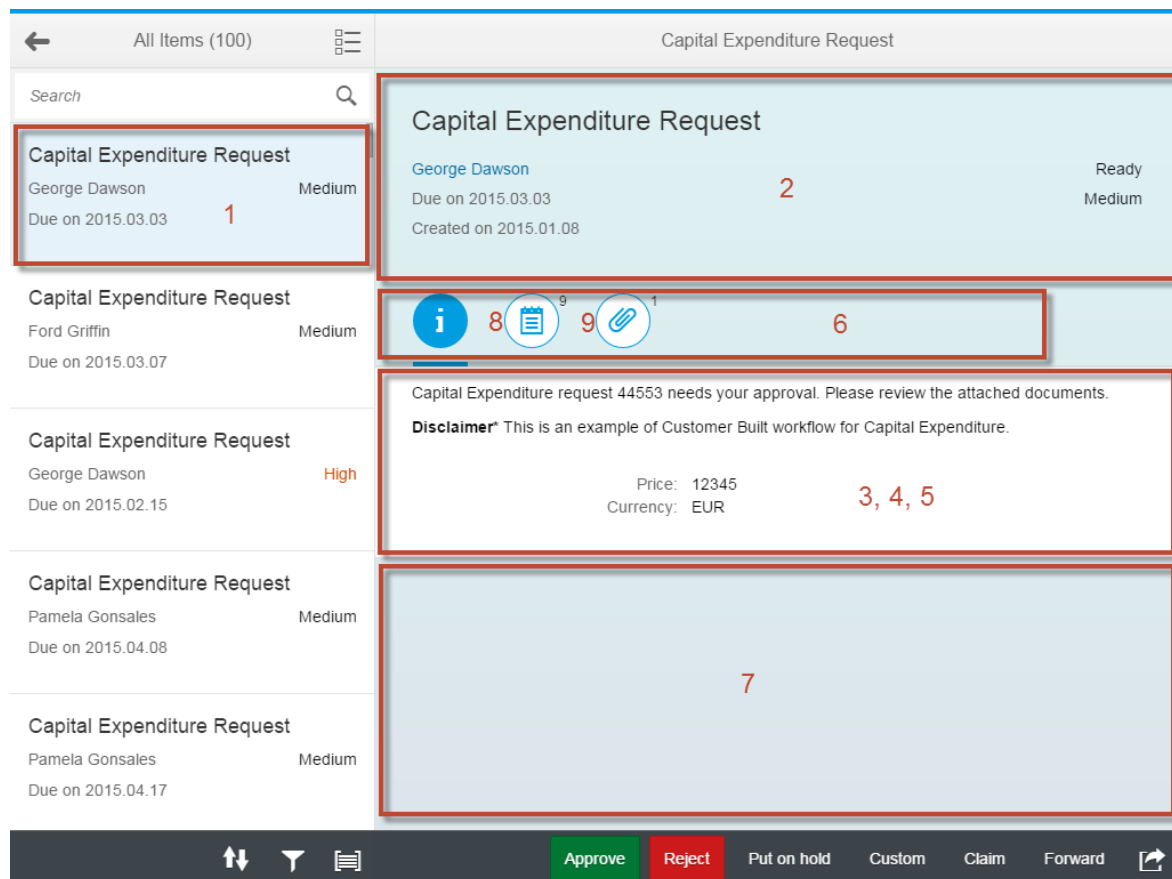
lv_workitem_id  = iv_work_item.
lv_approval_note = iv_approval_note.
lt_item_decision = it_item_decision.
```

```
CALL METHOD lo_api->set_decision_item_level
EXPORTING
    iv_workitem_id    = lv_workitem_id
    it_item_decision = lt_item_decision
    iv_approval_note = lv_approval_note
    io_msg_container = lo_msg_container.

ENDFUNCTION.
```

Appendix B - Extension Points

The following picture gives an overview about which extension points to use when you want to extend a specific section of the application screen:



S2.view.xml

1. CustomerExtensionForObjectListItem

S3.view.xml

2. CustomerExtensionForObjectHeader
3. CustomerExtensionForInfoTabContent
4. CustomerExtensionForNoteTabContent
5. CustomerExtensionForAttachmentTabContent
6. CustomerExtensionForAdditionalTabs
7. CustomerExtensionForAdditionalDetails

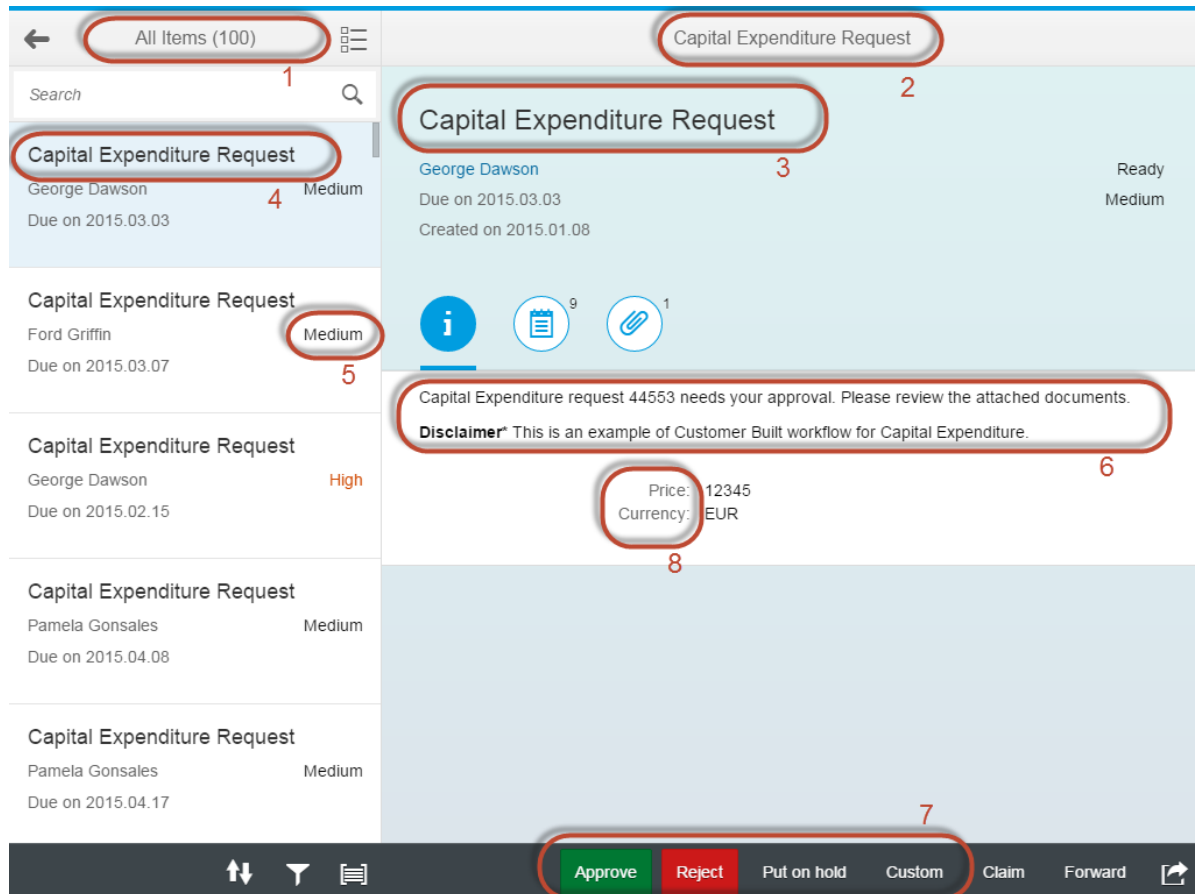
Appendix C - Controller Hooks

To plug in and execute custom code, the following hooks are available in the controller code:

Controller	Hook	Use
S2.controller.js	extHookGetCustomFilter	Allows you to create custom filters that can be used for filtering the workflow tasks
S2.controller.js	extHookChangeFilterItems	Allows you to replace the standard filters with custom filters based on the filter key
S2.controller.js	extHookChangeSortConfig	Allows you to change those properties that can be used for sorting the workflow tasks
S2.controller.js	extHookChangeGroupConfig	Allows you to change those properties that can be used for grouping the workflow tasks
S3.controller.js	extHookChangeFooterButtons	Allows you to add and change the action buttons on the detail screen
S2.controller.js	extHookChangeMassApprovalButtons	Allows you to add, remove, and change the action buttons on the list screen in mass action mode.
S3.controller.js	extHookGetEntitySetsToExpand	Allows you to retrieve additional entities from the back-end system
S3.controller.js	extHookOnDataLoaded	Allows you to perform additional requests and process the retrieved data
Forward.controller.js	extHookChangeListSizeLimit	Allows you to change the number of items shown in the forward screen.

Appendix D - Customizing the UI Fields

You can customize the following fields on the UI without creating an extended app:



Footnote

1) Application Title (Gateway)

The standard My Inbox has the [All Items](#) title.

To define a scenario-specific title, go to Customizing for SAP NetWeaver, choose Gateway Service Enablement ->Content ->Task Gateway Service ->Scenario Definition.

Enter a title under [Scenario Display Name](#).

To translate the application title to other languages, choose [Menu -> Goto -> Translation](#).

If the [Scenario Display Name](#) is empty, the default title is [My Inbox](#).

2.), 3), and 4) Task Title (Back End)

Start the Workflow Builder in the back-end system (transaction [SWDD](#)).

Enter your workflow ID, select the corresponding step, and change the task title.

To translate the task title to other languages, choose [Menu -> Extras -> Translation](#). This option is only available when the step details are closed.

5) Priority (Gateway)

In the `/IWPGW/V_TGW_FLT` maintenance view change the display name.

To translate the priority to other languages, choose [Menu -> Goto -> Translation](#)

If the display name is not defined, the values are hard-coded in the UI.

6) Description (Back End)

Start the Workflow Builder in the back-end system (transaction [SWDD](#)).

Enter your workflow ID and select the corresponding step. On the [Control](#) tab, double-click [Task](#). On the [Description](#) tab, change the description text.

To translate the description to other languages, change the language on the [Description](#) tab.

7) Decision Options (Back End)

In Customizing for SAP NetWeaver, choose Gateway Service Enablement -> Content -> Workflow Settings -> Maintain Task Names and Decision Options.

Enter your decision options under [Decision Text](#) on the [Decision Keys](#) tab.

To translate the decision options to other languages, choose [Menu -> Goto -> Translation](#).

If this customizing view is not set, you can define your decision options in the Service Builder (transaction [SWDD](#)): enter your workflow ID, select the corresponding step, and change the decision text on the [Decision](#) tab.

To translate the decision options to other languages, choose [Menu -> Extras -> Translation](#). This option is only available when the step details are closed.

In addition to the customizing options described above, you can change the following fields with BAdIs:

4) Task Title on the S2 Screen (Gateway)

Task Gateway: Change subject in query BAdI (`/IWPGW/BADI_TGW_TASK_QUERY`)

You can use this BAdI to change the task title for all the items in the list.

3) And 6) Task Title and Description on the S3 Screen (Gateway)

Task Gateway: Modify task data BAdI (`/IWPGW/BADI_TGW_TASK_DATA`)

You can use this BAdI to change the task title and the task description on the detail screen.

8) Custom Attributes (Back End)

BAdI Definition and Data for Custom Attributes (`/IWWRK/BADI_TGW_CUSTOM_ATTR`)

You can use this BAdI to display custom attributes for tasks retrieved from your Business Workflow.

Appendix E - Source Code of the Gateway Service (ZCL_Z_CARTAPPROVA_DPC_EXT)

ENTITYSET_TASK - Get list of tasks

METHOD entityset_task.

DATA:

```
    lo_tasks          TYPE REF TO /iwpgw/if_tgw_types=>tt_tasks,
    ls_task            TYPE /iwpgw/if_tgw_types=>ty_task,
    lt_tasks_ext       TYPE zcl_z_cartapprova_mpc_ext=>tt_task,
    ls_task_ext        TYPE zcl_z_cartapprova_mpc_ext=>ts_task,
*   structures for RFC
    lt_rfc_tasklist    TYPE zcartapp_t_apv_sc_wf_list,
    ls_rfc_task        TYPE zcartapp_s_apv_sc_wf_list.

* get work item details
CALL METHOD super->entityset_task

    RECEIVING

        rr_entityset = rr_entityset.

* call backend for custom field values
CALL FUNCTION 'ZCARTAPP_GET_WFTASKLIST' DESTINATION 'S3FCLNT300_T'

    IMPORTING

        et_list_data = lt_rfc_tasklist.

* map results
CREATE DATA lo_tasks.
lo_tasks ?= rr_entityset.
LOOP AT lo_tasks->* INTO ls_task.
    CLEAR ls_task_ext.
    MOVE-CORRESPONDING ls_task TO ls_task_ext.
```

```

    READ TABLE lt_rfc_tasklist INTO ls_rfc_task WITH KEY workitem_id = ls_task-
inst_id.

```

```

    ls_task_ext-value = ls_rfc_task-value.

```

```

    ls_task_ext-currency = ls_rfc_task-currency.

```

```

*    ls_task_ext-approval_on_itemlevel = ls_rfc_task-approval_on_itemlevel.

```

```

    IF ls_rfc_task-workitem_id+11(1) <= 5.

```

```

        ls_task_ext-approval_on_itemlevel = 'X'.

```

```

    ELSE.

```

```

        ls_task_ext-approval_on_itemlevel = ''.

```

```

    ENDIF.

```

```

    ls_task_ext-sc_number = ls_rfc_task-sc_number.

```

```

    APPEND ls_task_ext TO lt_tasks_ext.

```

```

ENDLOOP.

```

```

CALL METHOD copy_data_to_ref

```

```

    EXPORTING

```

```

        is_data = lt_tasks_ext

```

```

    CHANGING

```

```

        cr_data = rr_entityset.

```

```

ENDMETHOD.

```

CASE_EXPANDED_ENTITY - Case get expand entity

```

METHOD case_expanded_entity.

```

```

    DATA:

```

```

        lr_expanded_task_ext TYPE REF TO zcartapp_s_tgw_expand_deep,

```

```

        lr_exp_task          TYPE REF TO /iwpgw/wf_tgw_expand=>/iwwrk/s_tgw_expand
_deep,

```

```

        lr_entity            TYPE REF TO data,

```

```

        ls_scaitem           TYPE zcartapp_s_scaitem,

```

```

        lt_scaitem           TYPE zcartapp_t_scaitem,

```

```

        ls_scaapprover       TYPE zcartapp_s_scaapprover,

```

```

        lt_scaapprover       TYPE zcartapp_t_scaapprover,

```

```

        ls_key               TYPE /iwbep/s_mgw_name_value_pair,

```

```

        ls_key_item          TYPE /iwbep/s_mgw_name_value_pair,

```

```

*    variables for RFC

```

```

        lv_wiid              TYPE sww_wiid,

```

```

        ls_header            TYPE zcartapp_s_apv_sc_hd,

```

```

        ls_item                TYPE zcartapp_s_apv_sc_it,
        lt_item                TYPE zcartapp_t_apv_sc_it,
        ls_approver            TYPE zcartapp_s_apv_sc_approver,
        lt_approver            TYPE zcartapp_t_apv_sc_approver.

* get work item details
CALL METHOD super->case_expanded_entity
EXPORTING
    iv_entity_name            = iv_entity_name
    iv_entity_set_name        = iv_entity_set_name
    iv_source_name            = iv_source_name
    it_key_tab                = it_key_tab
    it_navigation_path        = it_navigation_path
    io_expand                 = io_expand
    io_tech_request_context   = io_tech_request_context
    it_req_expand             = it_req_expand
IMPORTING
    er_entity                 = lr_entity
    et_expanded_clauses       = et_expanded_clauses
    et_expanded_tech_clauses = et_expanded_tech_clauses.

* call backend for custom field values
READ TABLE it_key_tab INTO ls_key WITH KEY name = 'InstanceID'.
READ TABLE it_key_tab INTO ls_key_item WITH KEY name = 'ItemNumber'.
lv_wiid = ls_key-value.
CALL FUNCTION 'ZCARTAPP_GET_SC_DETAILS' DESTINATION 'S3FCLNT300_T'
EXPORTING
    iv_workitem_id = lv_wiid
IMPORTING
    es_header      = ls_header
    et_item        = lt_item
    et_approver    = lt_approver.

* map results
CREATE DATA lr_expanded_task_ext.
lr_exp_task ?= lr_entity.
MOVE-CORRESPONDING lr_exp_task->* TO lr_expanded_task_ext->*.

lr_expanded_task_ext->value = ls_header-value.
lr_expanded_task_ext->currency = ls_header-currency.
* lr_expanded_task_ext->approval_on_itemlevel = ls_header-approval_on_itemlevel.
lr_expanded_task_ext->sc_number = ls_header-sc_number.

IF lv_wiid+11(1) <= 5.
    lr_expanded_task_ext->approval_on_itemlevel = 'X'.
ELSE.
    lr_expanded_task_ext->approval_on_itemlevel = ''.
ENDIF.

LOOP AT lt_item INTO ls_item.
    ls_scaitem-instance_id = lv_wiid.
    ls_scaitem-item_number = ls_item-item_number.
    ls_scaitem-description = ls_item-description.
    ls_scaitem-vendor_id   = ls_item-vendor_id.
    ls_scaitem-vendor_name = ls_item-vendor_name.
    ls_scaitem-unit        = ls_item-unit.
    ls_scaitem-unit_text   = ls_item-unit_text.
    ls_scaitem-quantity    = ls_item-quantity.

```

```

ls_scaitem-value      = ls_item-value.
ls_scaitem-currency   = ls_item-currency.
ls_scaitem-price      = ls_item-price.
ls_scaitem-price_unit = ls_item-price_unit.
ls_scaitem-delivery_date = ls_item-delivery_date.
ls_scaitem-category_description = ls_item-category_description.
APPEND ls_scaitem TO lt_scaitem.
ENDLOOP.
lr_expanded_task_ext->scaitems = lt_scaitem.

LOOP AT lt_approver INTO ls_approver.
  ls_scaapprover-instance_id      = lv_wiid.
  ls_scaapprover-approver_id      = ls_approver-approver_id.
  ls_scaapprover-approver_sequence = ls_approver-approver_sequence.
  ls_scaapprover-approver_firstname = ls_approver-approver_firstname.
  ls_scaapprover-approver_lastname = ls_approver-approver_lastname.
  ls_scaapprover-approver_fullname = ls_approver-approver_fullname.
  ls_scaapprover-approver_role     = ls_approver-approver_role.
  ls_scaapprover-approval_status   = ls_approver-approval_status.
  ls_scaapprover-approval_statuscode = ls_approver-approval_statuscode.
  APPEND ls_scaapprover TO lt_scaapprover.
ENDLOOP.
lr_expanded_task_ext->scaapproversheader = lt_scaapprover.

er_entity = lr_expanded_task_ext.

ENDMETHOD.

```

/IWBEP/IF_MGW_APPL_SRV_RUNTIME~GET_ENTITY - Execute a READ request (CReadUD) - ENTRY -

METHOD /iwbeb/if_mgw_appl_srv_runtime~get_entity.

```

DATA:
  ls_scaitem      TYPE zcl_z_cartapprova_mpc=>ts_scaitem,
  ls_scaapprover  TYPE zcl_z_cartapprova_mpc=>ts_scaapprover,
  ls_key          TYPE /iwbeb/s_mgw_name_value_pair,
  ls_key_item     TYPE /iwbeb/s_mgw_name_value_pair,
* RFC structures
  lv_wiid        TYPE sww_wiid,
  ls_header      TYPE zcartapp_s_apv_sc_hd,
  ls_item        TYPE zcartapp_s_apv_sc_it,
  ls_approver    TYPE zcartapp_s_apv_sc_approver,
  lt_item        TYPE zcartapp_t_apv_sc_it,
  lt_approver    TYPE zcartapp_t_apv_sc_approver,
  lv_rfc_dest    TYPE string.

* handle custom and standard entities
CASE iv_entity_name.
  WHEN 'SCAItem'.
*   call backend for custom field values
  READ TABLE it_key_tab INTO ls_key WITH KEY name = 'InstanceID'.
  READ TABLE it_key_tab INTO ls_key_item WITH KEY name = 'ItemNumber'.
  lv_wiid = ls_key-value.
*   rfc_dest = get_system_alias_for_taskdef(
*     iv_scenario_id      = 'DEMO_CARTAPPROVAL'
*     iv_taskdefinition_id =

```

```

*      ).
      CALL FUNCTION 'ZCARTAPP_GET_SC_DETAILS' DESTINATION 'S3FCLNT300_T'
        EXPORTING
          iv_workitem_id = lv_wiid
        IMPORTING
          es_header      = ls_header
          et_item        = lt_item
          et_approver    = lt_approver.

*      map results
      READ TABLE lt_item INTO ls_item WITH KEY item_number = ls_key_item-value.
      ls_scaitem-instance_id = lv_wiid.
      ls_scaitem-item_number = ls_item-item_number.
      ls_scaitem-description = ls_item-description.
      ls_scaitem-vendor_id   = ls_item-vendor_id.
      ls_scaitem-vendor_name = ls_item-vendor_name.
      ls_scaitem-unit        = ls_item-unit.
      ls_scaitem-unit_text   = ls_item-unit_text.
      ls_scaitem-quantity    = ls_item-quantity.
      ls_scaitem-value       = ls_item-value.
      ls_scaitem-currency    = ls_item-currency.
      ls_scaitem-price       = ls_item-price.
      ls_scaitem-price_unit  = ls_item-price_unit.
      ls_scaitem-delivery_date = ls_item-delivery_date.
      ls_scaitem-category_description = ls_item-category_description.

      CALL METHOD copy_data_to_ref
        EXPORTING
          is_data = ls_scaitem
        CHANGING
          cr_data = er_entity.

      WHEN OTHERS.
*      handle standard entities
      CALL METHOD super->/iwbep/if_mgw_appl_srv_runtime~get_entity
        EXPORTING
          iv_entity_name      = iv_entity_name
          iv_entity_set_name   = iv_entity_set_name
          iv_source_name       = iv_source_name
          it_key_tab           = it_key_tab
          it_navigation_path    = it_navigation_path
          io_tech_request_context = io_tech_request_context
        IMPORTING
          er_entity            = er_entity.

      ENDCASE.

      ENDMETHOD.

```

/IWBEP/IF_MGW_APPL_SRV_RUNTIME~CREATE_DEEP_ENTITY - Execute a deep insert CREATE request (CreateRUD)

METHOD /iwbep/if_mgw_appl_srv_runtime~create_deep_entity.

```

      DATA: ls_header_approval TYPE zcartapp_s_scaheader_approval,
            lt_item_decision    TYPE zcartapp_t_scaitem_approval.

```



```

DATA: lv_workitem_id      TYPE sww_wiid,
      lv_approval_note    TYPE string.

CASE iv_entity_name.

    WHEN 'SCAHeaderApproval'.

        io_data_provider->read_entry_data( IMPORTING es_data = ls_header_approval ).

        lv_workitem_id    = ls_header_approval-workitem_id.
        lv_approval_note  = ls_header_approval-approval_note.
        lt_item_decision  = ls_header_approval-scaapprovalitems.

        CALL FUNCTION 'ZCARTAPP_ITEMLEVEL_APPROVAL' DESTINATION 'S3FCLINT300_T'
            EXPORTING
                iv_work_item      = lv_workitem_id
                iv_approval_note  = lv_approval_note
                it_item_decision  = lt_item_decision.

        IF sy-subrc EQ 0.
            copy_data_to_ref( EXPORTING is_data = ls_header_approval
                              CHANGING   cr_data = er_deep_entity ).
        ELSE.
            RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception.
        ENDIF.

    WHEN OTHERS.
        RETURN.

ENDCASE.

ENDMETHOD.

```

Appendix F - Source Code of the UI5 Application

Component.js

```

jQuery.sap.declare("cross.fnd.fiori.inbox_ext.Component");

cross.fnd.fiori.inbox_ext.sBaseAppUrl = "/sap/bc/ui5_ui5/sap/ca_fiori_inbox";

cross.fnd.fiori.inbox_ext.sServiceUrl = "/sap/opu/odata/sap/ZAREXT_CARTAPPROVAL_V2_SRV;mo/";

//Needed for fiori sandbox locally

//jQuery.sap.registerModulePath("cross.fnd.fiori.inbox", '/cross.fnd.fiori.inbox');

```

```
//Needed for fiori Launchpad
```

```
jQuery.sap.registerModulePath("cross.fnd.fiori.inbox", cross.fnd.fiori.inbox_ext.sBaseAppUrl);
```

```
jQuery.sap.require("cross.fnd.fiori.inbox.Component");
```

```
jQuery.sap.require("cross.fnd.fiori.inbox_ext.util.Formatter");
```

```
cross.fnd.fiori.inbox.Component.extend("cross.fnd.fiori.inbox_ext.Component",{
```

```
    metadata: {
```

```
        version : "1.0",
```

```
        config : {
```

```
            "sap.ca.serviceConfigs": [{
```

```
                name: "TASKPROCESSING",
```

```
                masterCollection: "TaskCollection",
```

```
                serviceUrl: cross.fnd.fiori.inbox_ext.sServiceUrl,
```

```
                isDefault: true,
```

```
                mockedDataSource: "/cross.fnd.fiori.inbox/model/metadata.xml"
```

```
            },
```

```
        {
```

```
            name: "POSTACTION",
```

```
            masterCollection: "TaskCollection",
```

```
            serviceUrl: cross.fnd.fiori.inbox_ext.sServiceUrl,
```

```
            isDefault: false,
```

```
            useBatch: true
```

```
        }}
```

```
    },
```

```
    customizing: {
```

```
        "sap.ui.viewExtensions": {
```

```
            "cross.fnd.fiori.inbox.view.S3": {
```

```
"CustomerExtensionForInfoTabContent": {
    className: "sap.ui.core.Fragment",
    fragmentName: "cross.fnd.fiori.inbox_ext.frag.InfoTab",
    type: "XML"
},
"CustomerExtensionForAdditionalDetails": {
    className: "sap.ui.core.Fragment",
    fragmentName: "cross.fnd.fiori.inbox_ext.frag.S3",
    type: "XML"
},
"CustomerExtensionForObjectHeader": {
    className: "sap.ui.core.Fragment",
    fragmentName:
"cross.fnd.fiori.inbox_ext.frag.S3_CustomerExtensionForObjectHeader",
    type: "XML"
},
"CustomerExtensionForNoteTabContent": {
    className: "sap.ui.core.Fragment",
    fragmentName: "cross.fnd.fiori.inbox_ext.frag.NoteTab",
    type: "XML"
},
"CustomerExtensionForAttachmentTabContent": {
    className: "sap.ui.core.Fragment",
    fragmentName: "cross.fnd.fiori.inbox_ext.frag.AttachmentTab",
    type: "XML"
},
"CustomerExtensionForAdditionalTabs": {
    className: "sap.ui.core.Fragment",
    fragmentName:
"cross.fnd.fiori.inbox_ext.frag.S3_CustomerExtensionForAdditionalTabs",
    type: "XML"
}
```

```

    },
    "cross.fnd.fiori.inbox.view.S2": {
        "CustomerExtensionForObjectListItem": {
            className: "sap.ui.core.Fragment",
            fragmentName:
"cross.fnd.fiori.inbox_ext.frag.S2_CustomerExtensionForObjectListItem",
            type: "XML"
        }
    }
},
"sap.ui.controllerExtensions": {
    "cross.fnd.fiori.inbox.view.S2": {
        "controllerName": "cross.fnd.fiori.inbox_ext.view.S2"
    },
    "cross.fnd.fiori.inbox.view.S3": {
        "controllerName": "cross.fnd.fiori.inbox_ext.view.S3"
    },
    "cross.fnd.fiori.inbox.view.Forward": {
        "controllerName": "cross.fnd.fiori.inbox_ext.view.Forward"
    }
}
},

routing : {
    routes: {
        masterDetail: {
            subroutes: {
                master: {
                    subroutes: {
                        "itemDetail": {
                            "pattern":
"itemDetail/{SAP__Origin}/{InstanceID}/{ItemNumber}",

```

```

"cross.fnd.fiori.inbox_ext.view",

"viewPath":

"view": "S4"

}

}

}

}

}

},

}

});

```

S2.controller.js

```

sap.ui.controller("cross.fnd.fiori.inbox_ext.view.S2", {

    extHookChangeFilterItems: function (aFilters) {

        var highPrioFilterItem = new sap.m.ViewSettingsFilterItem({

            text: 'Important',

            multiSelect: false

        });

        var filterValue = this._createFilterItem("Priority:HIGH_VERY_HIGH", "High and Very High");

        highPrioFilterItem.addItem(filterValue);

        aFilters.push(highPrioFilterItem);

        var valueFilterItem = new sap.m.ViewSettingsFilterItem({

            text: 'Value',

            multiSelect: false

        });
    }
});

```

```

        var filterValue = this._createFilterItem("Value:High", "1000 or more");

        valueFilterItem.addItem(filterValue);

        var filterValue = this._createFilterItem("Value:Low", "Less than 1000");

        valueFilterItem.addItem(filterValue);

        aFilters.push(valueFilterItem);

```

```

    },

```

```

    extHookChangeSortConfig: function (oSortConfig) {

        oSortConfig["Value"] = {

            text: "Amount",

            getVisible: function() { return true; }

        };

    },

```

```

    extHookChangeGroupConfig: function (aGroupConfig) {

        aGroupConfig.push({

            key: "Currency",

            textKey: "Currency",

            formatter: function(oContext) {

                switch(oContext.getProperty("Currency")){

                    case "EUR": {

                        return "Euro";

                        break;

                    }

                    case "USD": {

                        return "US Dollar";

                        break;

                    }

                }

            }

        });

    },

```

```

        default: {
            return oContext.getProperty("Currency");
            break;
        }
    }
}

});

},

extHookGetCustomFilter: function (oFilterKey) {

    var oFilters = new Array();

    if (oFilterKey["Priority:HIGH_VERY_HIGH"]){

        oFilters.push(new sap.ui.model.Filter("Priority", sap.ui.model.FilterOperator.EQ, "HIGH"));

        oFilters.push(new sap.ui.model.Filter("Priority", sap.ui.model.FilterOperator.EQ,
"VERY_HIGH"));

        return oFilters;

    }

    if (oFilterKey["Value:High"]){

        oFilters.push(new sap.ui.model.Filter("Value", sap.ui.model.FilterOperator.GE, 1000));

        return oFilters;

    }

    if (oFilterKey["Value:Low"]){

        oFilters.push(new sap.ui.model.Filter("Value", sap.ui.model.FilterOperator.LT, 1000));

        return oFilters;

    }

},

extHookChangeMassApprovalButtons: function (oButtonList) {

    var oButton = {

        sBtnTxt: "Extension Button",

        onBtnPressed: function() {

            alert("Extension button pressed!")

```

```

        }

    };

    oButtonList.aButtonList.push(oButton);

}

});

```

S3.controller.js

```

sap.ui.controller("cross.fnd.fiori.inbox_ext.view.S3", {

    _oApprovalItems: {},

    onShowItemDetails: function (oEvent) {

        var viewBindingContext = this.getView().getBindingContext();

        var itemBindingContext = oEvent.getSource().getBindingContext("detail");

        var sOrigin = viewBindingContext.getProperty("SAP__Origin");

        var sInstanceID = viewBindingContext.getProperty("InstanceID");

        var itemNumber;

        itemNumber = itemBindingContext.getProperty("ItemNumber");

        this.oRouter.navTo("itemDetail", {

            SAP__Origin: sOrigin,

            InstanceID: sInstanceID,

            ItemNumber: itemNumber

        });

    },

    extHookOnDataLoaded: function (oData) {

```



```
var i, imax, items, item;
```

```
this._oApprovalItems.WorkitemID = oData.InstanceID;
```

```
this._oApprovalItems.ScNumber = oData.ScNumber;
```

```
this._oApprovalItems.Comment = "";
```

```
this._oApprovalItems.SCAApprovalItems = [];
```

```
for (i = 0, items = oData.SCAItems.results, imax = items.length; i < imax; i++) {
```

```
    item = {};
```

```
    item.WorkitemID = items[i].InstanceID;
```

```
    item.ItemNumber = items[i].ItemNumber;
```

```
    item.ItemApproved = "X";
```

```
    this._oApprovalItems.SCAApprovalItems.push(item);
```

```
}
```

```
},
```

```
onItemApproveChanged: function (oEvent) {
```

```
    var i, imax;
```

```
    var itemNumber = oEvent.getSource().getBindingContext("detail").getProperty("ItemNumber");
```

```
    var itemApproved = oEvent.getParameter("state") ? "X" : "0";
```

```
    for (i = 0, imax = this._oApprovalItems.SCAApprovalItems.length; i < imax; i++) {
```

```
        if (this._oApprovalItems.SCAApprovalItems[i].ItemNumber === itemNumber) {
```

```
            this._oApprovalItems.SCAApprovalItems[i].ItemApproved = itemApproved;
```

```
            return;
```

```
        }
```

```
    }
```

```
},
```

```
extHookGetEntitySetsToExpand: function () {
```

```
    return ["SCAItems", "SCAApproversHeader"];
```

```
},
```

```
extHookChangeFooterButtons: function (oButtonList) {
```

```
    var bApproveOnItemLevel = this.getView().getBindingContext().getProperty("ApprovalOnItemlevel");
```

```
    //check if there is item level approval available
```

```
    if (bApproveOnItemLevel==true) {
```

```
        //remove Pos and Neg buttons and add Send to the first place
```

```
        oButtonList.oPositiveAction = null;
```

```
        oButtonList.oNegativeAction = null;
```

```
        var that = this;
```

```
        if (oButtonList.aButtonList) {
```

```
            oButtonList.aButtonList.unshift({
```

```
                sBtnTxt : "Send",
```

```
                onBtnPressed: function (event) {
```

```
                    that.showItemApprovalDialog();
```

```
                }
```

```
            });
```

```
        }
```

```
    }
```

```
},
```

```
showItemApprovalDialog: function () {
```

```
    sap.ca.ui.dialog.confirmation.open({
```

```
        question: "Send", //this.i18nBundle.getText("XMSG_DECISION_QUESTION", oDecision.DecisionText),
```

```
        showNote: true,
```

```
        title: this.i18nBundle.getText("XTIT_SUBMIT_DECISION"),
```

```
        confirmButtonLabel: this.i18nBundle.getText("XBUT_SUBMIT"),
```

```
        noteMandatory: false,
```

```

}, $.proxy(function (oResult) {

    if (oResult.isConfirmed===true){

        var viewBindingContext = this.getView().getBindingContext();

        var sOrigin = viewBindingContext.getProperty("SAP__Origin");

        var sInstanceID = viewBindingContext.getProperty("InstanceID");


        this.performCreate("SCAHeaderApprovalCollection", sOrigin, sInstanceID, this._getSCAHeaderApprovalCollection(),
$.proxy(function(oData) {

            // item is removed from S2 list in the data manager

            jQuery.sap.delayedCall(500, this, function(){

                sap.ca.ui.message.showMessageToast(this.i18nBundle.getText("dialog.success.complete"));

            });

            }, this), function(oError) {

                //error dialog is handled by ConnectionManager

            }

        );

    }

}, this)

);

},

```

```

performCreate: function (sPath, sOrigin, sInstanceID, oData, onSuccess, onError) {

    sap.ca.ui.utils.busydialog.requireBusyDialog(); //TODO: check why it does not show up

```

```

// we need to use the origin in the url, so own model is needed for the create

    var sPostServiceUrl = this.getView().getModel("POSTACTION").sServiceUrl;

    if (sPostServiceUrl.indexOf(";mo") != -1) {

        var urlParts = sPostServiceUrl.split(";mo");

        sPostServiceUrl = urlParts[0] + ";o=" + sOrigin + urlParts[1];

    }

    var oPostActionModel = new sap.ui.model.odata.ODataModel(sPostServiceUrl, true);

```

```

oPostActionModel.setUseBatch(true);

var oSettings = {
    success: $.proxy(function(oData, oResponse) {
        sap.ca.ui.utils.busydialog.releaseBusyDialog();
        $.sap.log.info("successful action");
        this.oDataManager.processListAfterAction(sOrigin, sInstanceID);
        if (onSuccess) {
            $.proxy(onSuccess(oData, oResponse), this);
        }
    }, this),
    error: $.proxy(function(oError) {
        sap.ca.ui.utils.busydialog.releaseBusyDialog();
        this.oDataManager.processListAfterAction(sOrigin, sInstanceID);
        if (onError)
            onError(oError);
    }, this),
    async: true
};

oPostActionModel.create(sPath, oData, oSettings);

},

```

```

_getSCAHeaderApprovalCollection: function () {
    return this._oApprovalItems;
},});

```

S4.controller.js

```

jQuery.sap.require("sap.ca.scfld.md.controller.BaseDetailController");

jQuery.sap.require("sap.ca.ui.utils.busydialog");

```

```

sap.ca.scfld.md.controller.BaseDetailController.extend("cross.fnd.fiori.inbox_ext.view.S4", {

    onInit: function() {

        // Execute onInit of base class.

        sap.ca.scfld.md.controller.BaseDetailController.prototype.onInit.call(this);


        // Setting the navigate back visible true for moving back to S3 controller.

        this.getView().byId("page").setShowNavButton(true);


        // Get DataManager instance.

        this.oDataManager = sap.ca.scfld.md.app.Application.getImpl().getComponent().getDataManager();


        this.oRouter.attachRouteMatched(function(oEvent) {

            if (oEvent.getParameter("name") == "itemDetail") {

                // This code will be executed when the user navigates to S4.


                // Read parameters.

                var args = oEvent.getParameter("arguments");

                var sOrigin = args.SAP__Origin;

                var sInstanceID = args.InstanceID;

                var sItemNumber = args.ItemNumber;


                // Execute line item query.

                this.readItem(sOrigin, sInstanceID, sItemNumber,

                            jQuery.proxy(this.readItemSuccess, this),

                            null);

            }

        }, this);

    },

    readItem: function(sOrigin, sInstanceID, sItemNumber, onSuccess, onError) {

        var that = this;

```

```

this.getView().setModel(new sap.ui.model.json.JSONModel());

sap.ca.ui.utils.busydialog.requireBusyDialog();

// Assemble GET request and execute it.

this.oDataManager.oDataRead("/SCAItemCollection(SAP__Origin=\"" + jQuery.sap.encodeURL(sOrigin) +
    "\",InstanceID=\"" + jQuery.sap.encodeURL(sInstanceID) +
    "\",ItemNumber=\"" + jQuery.sap.encodeURL(sItemNumber) + "\""),
    null,
    null,
    true,

    function(oData, oResponse) {

        sap.ca.ui.utils.busydialog.releaseBusyDialog();

        if (oData) {

            if (onSuccess)

onSuccess(oData);

        }

    },

    function(oError) {

sap.ca.ui.utils.busydialog.releaseBusyDialog();

that.oDataManager.oDataRequestFailed(oError);

        if (onError)

            onError(oError);

    });

},

readItemSuccess: function(oData) {

    // Set model of the view.

    var oModel = new sap.ui.model.json.JSONModel(oData);

    this.getView().setModel(oModel);

```

```

    }
});

```

S4.view.xml

```

<sap.ui.core:View controllerName="cross.fnd.fiori.inbox_ext.view.S4" xmlns="sap.m" xmlns:sap.ui.core="sap.ui.core"
xmlns:sap.ui.layout.form="sap.ui.layout.form" xmlns:sap.ui.layout="sap.ui.layout">

    <Page id="page" title="{/Description}">

        <content>

            <ObjectHeader title="{/Description}" number="{/Value}" numberUnit="{/Currency}">

                <attributes>

                    <ObjectAttribute text="{parts: [{path: '/Quantity'}, {path: '/UnitText'}],
formatter: 'cross.fnd.fiori.inbox_ext.util.Formatter.formatQuantity'}/>

                    <ObjectAttribute text="{parts: [{path: '/Price'}, {path: '/Currency'}, {path:
'/PriceUnit'}, {path: '/UnitText'}], formatter: 'cross.fnd.fiori.inbox_ext.util.Formatter.formatPrice'}/>

                </attributes>

            </ObjectHeader>

            <sap.ui.layout.form:SimpleForm minWidth="1024" editable="false">

                <sap.ui.layout.form:content>

                    <sap.ui.core:Title text="Information"/>

                    <Label text="Product">

                        <layoutData>

                            <sap.ui.layout:ResponsiveFlowLayoutData
minWidth="192" weight="3"/>

                        </layoutData>

                    </Label>

                    <Text text="{/Description}">

                        <layoutData>

                            <sap.ui.layout:ResponsiveFlowLayoutData
weight="5"/>

                        </layoutData>

                    </Text>

```

```

        <Label text="Product Category">
            <layoutData>
                <sap.ui.layout:ResponsiveFlowLayoutData
minWidth="192" weight="3"/>
            </layoutData>
        </Label>
        <Text text="{/CategoryDescription}">
            <layoutData>
                <sap.ui.layout:ResponsiveFlowLayoutData
weight="5"/>
            </layoutData>
        </Text>
    </sap.ui.layout.form:content>
</sap.ui.layout.form:SimpleForm>
</content>
</Page>
</sap.ui.core:View>

```

S2_CustomerExtensionForObjectListItem.fragment.xml

```

<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
    <ObjectListItem id="MAIN_LIST_ITEM" type="{device}/listItemType" press="_handleItemPress"
        title="{TaskTitle}" number="{Value}" numberUnit="{Currency}">
        <attributes>
            <ObjectAttribute id="ATTR1" text="{CreatedByName}" />
            <ObjectAttribute id="ATTR2" text="{path: 'IsEscalated', formatter:
'cross.fnd.fiori.inbox.Conversions.formatterEscalationState'}" />
        </attributes>
        <firstStatus>
            <ObjectStatus id="STAT1"
                text="{parts: [{path: 'SAP__Origin'}, {path: 'Priority'}], formatter:
'cross.fnd.fiori.inbox.Conversions.formatterPriority'}"

```



```

state="{path: 'Priority',
formatter:'cross.fnd.fiori.inbox.Conversions.formatterPriorityState'}">

    </ObjectStatus>

</firstStatus>

<secondStatus>

    <ObjectStatus id="STAT2" text="{path: 'CompletionDeadLine',
type:'sap.ca.ui.model.type.Date', formatOptions: {style:'daysAgo'}}" />

</secondStatus>

</ObjectListItem>
</core:FragmentDefinition>

```

S3_CustomerExtensionForAdditionalTabs.fragment.xml

```

<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

    <IconTabFilter id="SCAHeaderApprovalChain"

        icon="sap-icon://group"

        visible="{path: 'detail>/SCAApproversHeader/results/length', formatter:
'cross.fnd.fiori.inbox.Conversions.formatterIsNotZero' }">

        <List inset="true" showSeparators="None" items="{detail>/SCAApproversHeader/results}">

            <items>

                <StandardListItem title="{detail>ApproverFullName}" type="Active"
iconInset="false" press=""></StandardListItem>

            </items>

        </List>

    </IconTabFilter>
</core:FragmentDefinition>

```

S3_CustomerExtensionForObjectHeader.fragment.xml

```

<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

    <ObjectHeader title="{TaskTitle}" number="{Value}" numberUnit="{Currency}">

        <attributes>

            <ObjectAttribute id="ARUserName" text="{CreatedByName}" active="true"
press="onEmployeeLaunchTask" />

            <ObjectAttribute text="{path: 'IsEscalated', formatter:
'cross.fnd.fiori.inbox.Conversions.formatterEscalationState'}" />

        </attributes>

    </ObjectHeader>
</core:FragmentDefinition>

```

```

</attributes>

<firstStatus>

    <ObjectStatus

        text="{parts: [{path: 'SAP__Origin'}, {path: 'Priority'}], formatter:
'cross.fnd.fiori.inbox.Conversions.formatterPriority'}"

        state="{path: 'Priority',
formatter:'cross.fnd.fiori.inbox.Conversions.formatterPriorityState'}">

    </ObjectStatus>

</firstStatus>

<secondStatus>

    <ObjectStatus text="{path: 'CompletionDeadLine', type:'sap.ca.ui.model.type.Date',
formatOptions: {style:'daysAgo'}}"/>

</secondStatus>

</ObjectHeader>

</core:FragmentDefinition>

```

S3.fragment.xml

```

<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

    <Table items="{detail}/SCAItems/results">

        <headerToolbar>

            <Toolbar>

                <Label text="Items"/>

            </Toolbar>

        </headerToolbar>

        <columns>

            <Column>

            </Column>

            <Column>

                <Label text="Description"/>

            </Column>

        </columns>

    </Table>

</core:FragmentDefinition>

```

```

        <Column>

            <Label text="Quantity"/>

        </Column>

        <Column>

            <Label text="Subtotal"/>

        </Column>

    </columns>

    <items>

        <ColumnListItem

            type="Navigation"

            press="onShowItemDetails">

                <cells>

                    <Switch type="AcceptReject" state="true"
change="onItemApproveChanged" visible="{detail>/ApprovalOnItemlevel}"/>

                    <ObjectIdentifier title="{detail>Description}"/>

                    <ObjectNumber number="{detail>Quantity}" unit="{detail>UnitText}"/>

                    <ObjectNumber number="{detail>Price}" unit="{detail>Currency}"/>

                </cells>

            </ColumnListItem>

        </items>

    </Table>

</core:FragmentDefinition>

```

AttachmentTab.fragment.xml

```

<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

    <List inset="true" showSeparators="None" items="{detail>/Attachments/results}">

        <items>

            <StandardListItem

                title="{detail>FileName}"

                description="{detail>CreatedByName}"

                info="{path: 'detail>CreatedAt', type: 'sap.ca.ui.model.type.DateTime',
formatOptions: { style: 'short'}}"

```

```
icon="{path: 'detail>mime_type', formatter:
'sap.ca.ui.model.format.FormattingLibrary.formatAttachmentIcon'}"
```

```
type="Active"
```

```
iconInset="false"
```

```
press="onAttachmentShow">
```

```
</StandardListItem>
```

```
</items>
```

```
</List>
```

```
</core:FragmentDefinition>
```

InfoTab.fragment.xml

```
<core:FragmentDefinition
```

```
xmlns="sap.m"
```

```
xmlns:form="sap.ui.layout.form"
```

```
xmlns:layout="sap.ui.layout"
```

```
xmlns:core="sap.ui.core">
```

```
<form:Form>
```

```
<form:formContainers>
```

```
<form:FormContainer>
```

```
<form:formElements>
```

```
<form:FormElement>
```

```
<form:label>
```

```
<Label text="{i18n>view.Information.createdOn}"
```

```
design="Bold">
```

```
<layoutData>
```

```
<layout:ResponsiveFlowLayoutData minWidth="192" weight="3"></layout:ResponsiveFlowLayoutData>
```

```
</layoutData>
```

```
</Label>
```

```
</form:label>
```

```
<form:fields>
```

```

<Text text="{path: 'detail>/CreatedOn',
type:'sap.ca.ui.model.type.Date', formatOptions : { style:'long'}}">

    <layoutData>

<layout:ResponsiveFlowLayoutData weight="5"></layout:ResponsiveFlowLayoutData>

    </layoutData>

</Text>

</form:fields>

</form:FormElement>

<form:FormElement>

    <form:layoutData>

        <layout:ResponsiveFlowLayoutData linebreak="true"
margin="false"></layout:ResponsiveFlowLayoutData>

    </form:layoutData>

    <form:label>

        <Label text="{i18n>view.Information.description}"
design="Bold">

            <layoutData>

<layout:ResponsiveFlowLayoutData minWidth="192" weight="3"></layout:ResponsiveFlowLayoutData>

            </layoutData>

        </Label>

    </form:label>

    <form:fields>

        <core:HTML content="{path:
'detail>/Description/Description', formatter: 'cross.fnd.fiori.inbox.Conversions.formatterHtml'}">

            <core:layoutData>

<layout:ResponsiveFlowLayoutData weight="5"></layout:ResponsiveFlowLayoutData>

            </core:layoutData>

        </core:HTML>

    </form:fields>

</form:FormElement>

<form:FormElement>

```

```

        <form:layoutData>
            <layout:ResponsiveFlowLayoutData linebreak="true"
margin="false"></layout:ResponsiveFlowLayoutData>
        </form:layoutData>
        <form:label>
            <Label text="Status" design="Bold">
                <layoutData>
                    <layout:ResponsiveFlowLayoutData minWidth="192" weight="3"></layout:ResponsiveFlowLayoutData>
                </layoutData>
            </Label>
        </form:label>
        <form:fields>
            <Text text="{path: 'detail'}/Status'">
                <layoutData>
                    <layout:ResponsiveFlowLayoutData weight="5"></layout:ResponsiveFlowLayoutData>
                </layoutData>
            </Text>
        </form:fields>
    </form:FormElement>
</form:formElements>
</form:FormContainer>
</form:formContainers>
<form:layout>
    <form:ResponsiveLayout></form:ResponsiveLayout>
</form:layout>
</form:Form>
</core:FragmentDefinition>

```

NoteTab.fragment.xml

```

<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">

```

```

<List inset="true" showSeparators="None" items="{detail>/Comments/results}">

    <items>

        <FeedListItem id="ARcomments"

            text="{detail>Text}"

            sender="{path: 'detail>CreatedByName'}"

            senderPress="onEmployeeLaunchCommentSender"

            timestamp="{path: 'detail>CreatedAt', type: 'sap.ca.ui.model.type.DateTime',
formatOptions: { style: 'long'}}"

            icon="sap-icon://customer"

            iconPress="onEmployeeLaunchCommentIcon">

        </FeedListItem>

    </items>

</List>

</core:FragmentDefinition>

```

Formatter.js

```

jQuery.sap.declare("cross.fnd.fiori.inbox_ext.util.Formatter");

cross.fnd.fiori.inbox_ext.util.Formatter = {

    formatQuantity: function (quantity, unitText) {

        if (quantity != null && unitText != null) {

            return quantity + " " + unitText;

        } else {

            return "";

        }

    },

    formatPrice: function (price, currency, priceUnit, unitText) {

        if (price != null && currency != null && priceUnit != null && unitText != null) {

            return price + " " + currency + "/" + priceUnit + " " + unitText;

        } else {

```

```
        return "";
    }
}
};
```

Forward.Controller.js

```
sap.ui.controller("cross.fnd.fiori.inbox_ext.view.Forward", {

    extHookChangeListSizeLimit: function() {

        // limit the number of agents to 5

        return 5;

    }

});
```


www.sap.com/contactsap

© 2020 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

The information contained herein may be changed without prior notice. Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platforms, directions, and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, and they should not be relied upon in making purchasing decisions.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. All other product and service names mentioned are the trademarks of their respective companies.

